

ÖLÇÜLMÜŞ ÇALIŞMA NOTLARI · AĞUSTOS 2026

Bütün yüzey

AutoGen core · AgentChat · OpenClaw · ve bir enterprise asistanın buradan ne alabileceği

vc-agent · AutoGen v0.7.5 · OpenClaw @01cc7106

her iddia etiketli: ölçüldü · kaynak · teyitsiz

Bu deste ne, ve neden böyle sıralı

Bir çerçeve tanıtımı değil. AutoGen'in ve OpenClaw'ın **ne yaptığını** anlatan sayfalar zaten var: 34 sayfalık bir PDF, 1.18 MB resmî belge, iki Türkçe kılavuz. Onları tekrar etmenin bir değeri yok.

Bu deste, o belgeleri okuyup **kod yazarken çarptığımız** şeylerin destesi. Bu yüzden sıralama pedagojik değil, **maliyet sırasına** göre: önce bilmezsen seni yakan şeyler, sonra bilmek güzel olanlar.

Somut örnek: `max_tool_iterations` 'ın varsayılanı, AgentChat kılavuzunun 2.000. satırında geçen bir parametre. Bizde ikinci sıraya yakın, çünkü onu bilmeden kurulan her ajan **yanlış cevap veriyor** ve hiçbir yerde hata çıkmıyor.

Üç etiket var ve ciddiye alınmaları gerekiyor:

ÖLÇÜLDÜ bir koşudan veya kaynak koddan çıkan sayı. Yeniden üretilebilir — nasıl ölçüldüğü slaytta ya da dipnotta yazıyor.

KAYNAK bir belge böyle diyor; atıf slaytta. Doğruluğunu biz değil o belge üstleniyor.

TEYİTSİZ inanıyoruz, ölçmedik. Destedeki en tehlikeli etiket bu — çünkü bir slaytı kimse durdurup kontrol etmez. Bu yüzden ayrı bir renkle duruyor.

Tez – tek slaytta

Üç sistemi yan yana koyunca ortaya çıkan tek cümle:

Ajan döngüsü artık ilginç değil. Model çağır, tool çağır, sonucu geri ver — bunu herkes yapıyor ve AutoGen'de zaten hazır. Ayrımı yaratan şey **döngüyü kuşatan kontrol düzlemi**: neyin çağrılabilirdiği, kimin onayladığı, neyin kaydedildiği — ve bir mekanizmanın **neyi kanıtlamadığının** yazılı olması.

AutoGen motoru veriyor: aktör modeli, takımlar, akış, sonlandırma. Kontrol düzlemi **yok** — "kapı" diye bir kavramı yok. Bir ajanın dış dünyaya yaptığı çağrışı durdurup insana sormak istiyorsan, onu kendin kurman gerekiyor.

Bizim yaptığımız ikisini birleştirmek oldu. Atlas için önerilen de bu: motoru bir yerden, kuşatmayı başka yerden almak.

OpenClaw tam tersi: kontrol düzlemi zengin — onay, kapsam, denetim, sandbox, dış içerik sınırı. Ama ajan döngüsünün kendisi sıradan; AutoGen'in takım soyutlamalarına karşılık gelen bir şey yok.

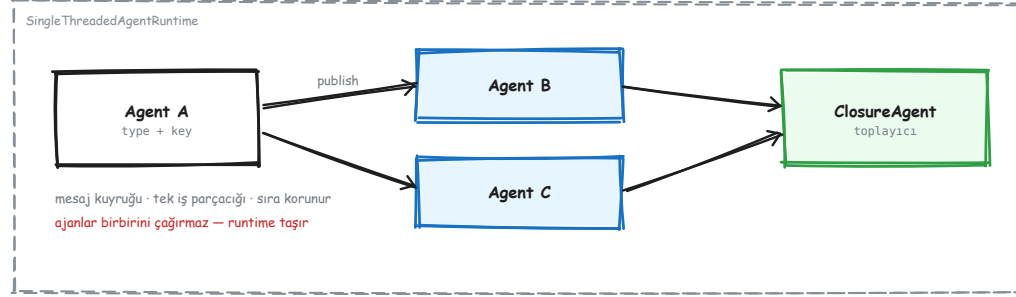


AutoGen core

Aktör modeli, kimlik, topic, yayın — ve dört tuzak

1. Aktör modeli ve runtime
2. Ajan kimliği: type + key
3. İki iletişim biçimi ve asimetrileri
4. Topic, abonelik, source→key kuralı
5. Fan-out / fan-in ve ClosureAgent
6. Müdahale ve gözlemlenebilirlik
7. Mesaj sözleşmesi, dağıtık runtime, protokoller
8. Dokuz desen ve ölçülen maliyetleri

Aktör modeli – neden böyle kurulmuş



Runtime mesajı taşır; ajanlar birbirinin referansını tutmaz.

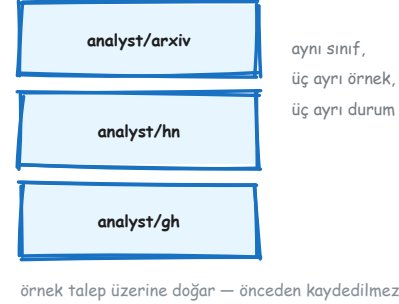
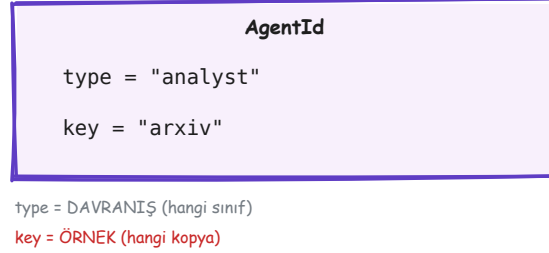
Ajanlar birbirini **çağırıyor**. Bir ajan başka bir ajanın nesnesini elinde tutmuyor, metodunu çağırıyor. Runtime'a bir mesaj veriyor; teslimatı runtime yapıyor.

Bunun bir bedeli var: araya bir dolaylılık katmanı giriyor ve "kim kimi çağırdı" sorusu artık yığın izinden okunmuyor.

Karşılığında üç şey geliyor:

- **Ajan eklemek çağıran kodu değiştirmiyor.** Dördüncü bir analist eklediğinde koordinatör dosyasına dokunmuyorsun — yeni ajan aynı topic'e abone oluyor, o kadar.
- **Teslimat noktası tek.** Bütün mesajlar tek bir yerden geçtiği için müdahale, log ve ölçüm oraya takılıyor.
- **Aynı sınıftan çok örnek bedava** — bir sonraki slaytın konusu.

Ajan kimliği – bir şey değil, iki şey



KAYNAK AgentId = (type, key). Çoğu core hatası bu ikisini tek şey sanmaktan çıkıyor.

type **davranış**: hangi sınıf, hangi handler'lar. Kayıt (register) bu düzeyde yapılır.

key **örnek**: aynı davranışın hangi kopyası, hangi durumu taşıyor. Kaydedilmez.

Örnekler **talep üzerine** doğuyor. analyst/hn'e ilk mesaj gittiğinde runtime o örneği yaratıyor, fabrika fonksiyonunu çağırıyor, sonra teslim ediyor.

Yani "ajanı kaydetmedim ama mesaj gitti" diye bir durum yok; ama "üç ajan kaydettim" de yanlış bir cümle — **bir tip** kaydettin, üç örnek doğdu.

Pratik sonuç: durumu key taşır. Aynı key'e iki kez mesaj gönderirsen aynı örneğe, aynı belleğe gider. Farklı key demek sıfırdan bir ajan demektir.

İki iletişim biçimi – ve simetrik olmadıkları

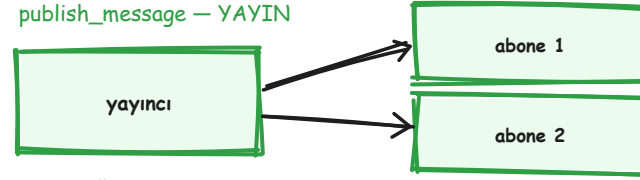
send_message — DOĞRUDAN



- cevabı DÖNDÜRÜR
- hata **ÇAĞIRANA** fırlar
- tek alıcı, bilinen adres

Bu asimetri ölçüldü: yayında düşen bir handler sessizce düşer.

publish_message — YAYIN



- None DÖNER — cevap yok
- hata yalnız **LOGLANIR**
- 0 abone de geçerli bir sonuç

`send_message` bir **fonksiyon çağırısı gibidir**: tek alıcı, dönüş değeri var, hata çağırana fırlar. Bir ajandan somut bir cevap bekliyorsan bu.

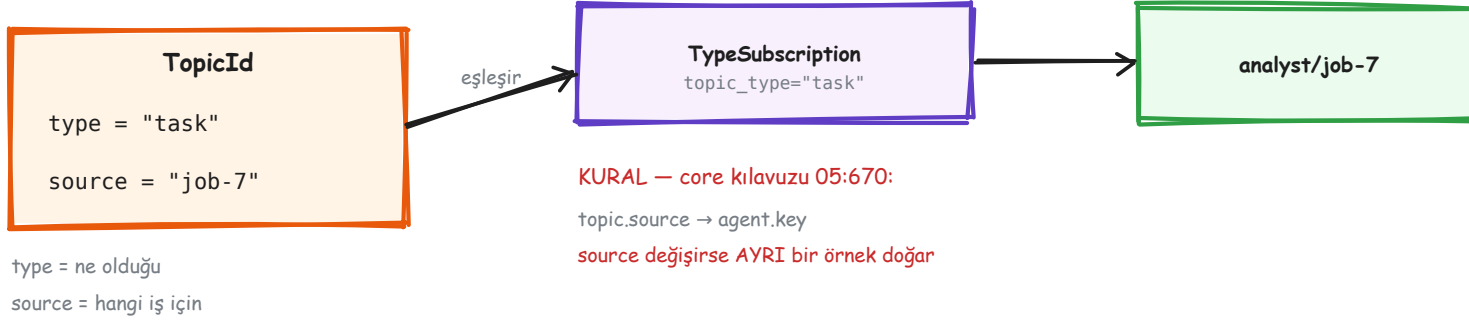
`publish_message` bir **duyurudur**: kaç dinleyici olduğunu bilmezsin, dönüş değeri yoktur, ve handler içinde patlayan istisna sana ulaşmaz — yalnız loglanır.

ÖLÇÜLDÜ Bu asimetriyi ölçtük. Yayın yolunda handler'ın attığı istisna çağırana **ulaşmıyor**; aynı istisna doğrudan mesajda çağırana fırlıyor.

Sıfır abone de geçerli bir sonuç: kimse dinlemiyorsa `publish_message` sessizce başarılı olur. Yani "aboneliği yazmayı unuttum" hatası da sessizdir.

Sonuç: fan-out mimarisinde bir dal sessizce ölebilir ve toplayıcı sonsuza kadar bekler. Bu yüzden sayaç **hata durumunda da** ilerlemek zorunda — Kısım III'te kodu var.

Topic nedir – gerçekten



Topic bir **kanal adı değil**, iki parçalı bir adres. `type` ne olduğunu söyler ("bu bir görev"), `source` hangi iş için olduğunu ("7 numaralı koşu").

Abonelik `type` 'a yapılır: `TypeSubscription("task", "analyst")` demek "task tipindeki her yayını analyst tipine ver" demektir.

KAYNAK Kural — core kılavuzu 05:670: teslim edilen örneğin `key` 'i, topic'in `source` 'undan gelir. Doğrudan, dönüşümsüz.

Yani `source` 'u her istekte değiştirirsen her istekte **yeni bir ajan örneği** doğar. Önceki örneğin biriktirdiği durum kaybolmaz — erişilemez hale gelir, ki bu daha kötüdür.

En sık görülen core hatası bu, ve hata vermez. Sistem çalışır, ajanlar cevap verir, sadece hiçbirini bir öncekini hatırlamaz.

publish anını kim belirler – ve parametreleri kim verir

Kimse otomatik yapmıyor. **Yazan kişi** belirliyor: kodunda `await runtime.publish_message(msg, TopicId(...))` satırını nereye koyduysan, publish anı orasıdır.

Parametreleri de yazan kişi veriyor: mesaj nesnesinin içeriği, topic type'ı, source'u. Framework hiçbirini türetmiyor, tahmin etmiyor, varsayılan atamıyor.

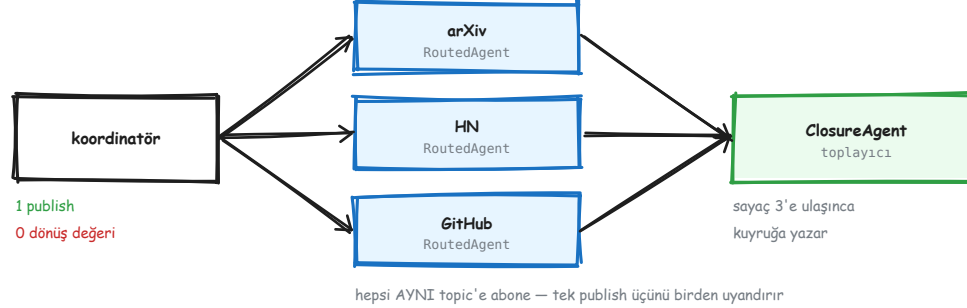
Bir handler'ın içinden yayın yapılabilir — o zaman zincir kurulur: A biter, B'yi tetikler, B biter, C'yi tetikler. Ama bu zinciri de yazan kişi kuruyor.

Bu soru bize üç farklı biçimde soruldu ve cevabı hep aynı çıktı: **AutoGen'de örtük hiçbir şey yok.** "Kim tetikledi" sorusunun cevabı her zaman senin yazdığın satırdır.

Bu, LangGraph'tan temel farkı. LangGraph'ta kenarlar tetikler — grafiği çizersin, çalıştırma sırasını çerçeve türetir. AutoGen'de grafiği **sen koşturursun.**

Bedeli: daha çok kod. Karşılığı: sürpriz yok.

Fan-out / fan-in – eşzamanlılık nereden geliyor



Tek publish üç ajanı birden uyandırır; toplama ayrı bir ajandır.

KAYNAK Eşzamanlılık ajanlardan gelmiyor, **abonelikten** geliyor. Üç analist de aynı topic tipine abone olduğu için tek bir yayın üçünü birden uyandırıyor; runtime üçünü paralel koşturuyor.

Yani "paralel çalıştır" diye bir çağrı yok. Paralellik, aynı duyuruyu birden fazla kişinin dinlemesinin doğal sonucu.

Toplama için **ayrı bir ajan** gerekiyor ve bunun sebebi mekanik: `publish_message` hiçbir şey döndürmüyor. Dönüş değeri olmayan bir çağrıdan sonuç toplayamazsın.

O yüzden analistler sonuçlarını **ikinci bir topic'e** yayınlıyor, toplayıcı ona abone oluyor ve kaç tane beklediğini kendisi sayıyor.

Sayacı **sen** tutuyorsun. Framework "üç dal vardı, üçü de bitti" demiyor — kaç dal olduğunu bilmiyor bile.

ClosureAgent – sınıf yazmadan ajan

```
async def collect(ctx, message, cx):  
    await queue.put(message)  
  
await ClosureAgent.register_closure(  
    runtime, "collector", collect,  
    subscriptions=lambda: [  
        TypeSubscription("result", "collector")  
    ],  
)
```

Toplayıcının bütün davranışı tek bir fonksiyon kadarsa, ona bir sınıf yazmak gürültüdür.

`ClosureAgent` bunu kapatıyor: bir fonksiyon veriyorsun, o ajan oluyor.

Tipik kullanım: sonuçları bir `asyncio.Queue` 'ya yazmak ve runtime'ın dışındaki kodun oradan okuması. Bu, aktör dünyasından normal async dünyaya çıkışın en temiz kapısı.

TEYİTSİZ Dikkat edilecek yer: runtime kapanışı ile kuyruk tüketimi arasındaki yarış sen yönetiyorsun. `stop_when_idle()` döndüğünde kuyrukta okunmamış eleman kalabilir.

Müdahale – kapınının AutoGen'deki tek atası



`InterventionHandler` runtime'a takılıyor. Her `on_send` ve `on_publish` ondan geçiyor — yani sistemdeki **bütün** mesaj trafiği tek bir noktadan görünüyor.

`DropMessage` döndürürse mesaj yok oluyor. Alıcı hiç haberdar olmuyor; gönderen de bir hata almıyor.

Bu, AutoGen'in onaya en yakın parçası. Ama **onay değil**: insana sormuyor, gerekçe döndürmüyor, kaydı yok, ve kararı geri bildirmiyor. Sadece bir süzgeç.

Bizim kapımız bu yüzden burada değil, bir katman yukarıda — `workbench` düzeyinde kuruldu. Orada tool'un adı, argümanları ve bir gerekçe döndürme imkânı var (Kısım IV).

Gözlemlenebilirlik – hazır gelen kısım

```
import logging
from autogen_core import EVENT_LOGGER_NAME

logging.getLogger(EVENT_LOGGER_NAME)\
    .addHandler(MyHandler())
```

AutoGen yapılandırılmış olayları **zaten** yayıyor; tek yapman gereken standart `logging` üstünden dinlemek.

`LLMCallEvent` — istem ve tamamlama token sayıları

`LLMStreamStartEvent` / `LLMStreamEndEvent`

`ToolCallEvent` — hangi tool, hangi argümanlarla

ÖLÇÜLDÜ Bizim maliyet muhasebemiz tamamen bunun üstünde. Ayrı bir sayaç yazmadık; token sayıları modelden geldiği gibi kaydediliyor, tahmin edilmiyor.

Bir çerçevenin ölçülebilir olması, ölçüm kodu yazmak zorunda kalmamak demektir. Burada olay akışı hazır — kullanmamak bir tercih, eksiklik değil.

Mesaj sözleşmesi ve serileştirme

```
@dataclass
class Signal:
    source: str
    url: str
    score: float
```

Mesajlar düz veri sınıfları — `dataclass` ya da pydantic modeli. Runtime onları serileştirebilmek zorunda, çünkü dağıtık runtime'da süreç sınırını geçiyorlar.

KAYNAK Tek süreçte serileştirme yapılmıyor (gereksiz kopya olurdu), ama **sözleşme aynı**. Yani tek süreçte doğru yazılmış bir tasarım dağıtığa taşındığında mesaj tarafında hiçbir şey değişmiyor.

Pratik kural: mesajın içine canlı nesne koyma — açık dosya tanıtıcısı, veritabanı bağlantısı, model istemcisi. Serileşemeyen bir mesaj tek süreçte çalışır, dağıtıktaki patlar, ve patladığı yer taşındığının gündür.

Tek süreç ve dağıtık runtime

	SINGLETHREADEDAGENTRUNTIME	GRPCWORKERAGENTRUNTIME
nerede koşar	tek süreç, tek iş parçacığı	süreç/makine başına bir worker
taşıma	bellek içi kuyruk	gRPC + bir host süreci
ne zaman	geliştirme ve çoğu üretim	ajanlar ayrı ölçeklenmeli
ajan kodu	aynı	aynı — yalnız kayıt ve başlatma değişir

KAYNAK Aynı ajan sınıfı ikisinde de değişmeden koşuyor. Ayrım kayıt ve başlatmada — davranışta değil. Bu, aktör modelinin asıl kazandırdığı şey.

Pratikte tek süreçle başlamak için sebep yok. Dağıtık runtime bir **ölçek cevabı**; ölçek problemi ölçülmeden ödenmemesi gereken bir karmaşıklık.

Konuştuğu protokoller

PROTOKOL	NE ÇÖZER	AUTOGEN'DEKİ YERİ
MCP	tool sunucusu ↔ istemci	<code>McpWorkbench</code> — birinci sınıf
A2A	ajan ↔ ajan, kurumlar arası	dışarıdan; çekirdekte yok
ACP	ajan ↔ istemci oturumu	dışarıdan
OpenAI uyumlu HTTP	model çağrısı	<code>OpenAIChatCompletionClient</code>

KAYNAK Bizim kullandığımız MCP. OpenClaw köprüsü de onun üstünde ve **iki yönlü**: onların tool'ları bize geliyor, bizim tool'larımız onlara gidiyor. Aynı protokol iki yöne de çalışıyor.

A2A ve ACP'nin bu projede karşılığı yok. Olmadığını söylemek, “destekliyor” demekten daha yararlı — çünkü ikincisi denemeye çağırır ve deneme boşa gider.

Rakiplerle – ne zaman hangisi

ÇERÇEVE	GÜÇLÜ YANI	BİZİM İÇİN SORUNU
AutoGen	aktör modeli, takımlar, olay akışı	bakım modu okuması
LangGraph	durum makinesi, kalıcılık, tekrar oynatma	grafik dili ağır, öğrenme eğrisi dik
CrewAI	hızlı başlangıç, rol metaforu	kontrol az; kapı/politika kavramı yok
OpenAI Agents SDK	sadelik, az kavram	tek sağlayıcıya yakın duruyor
Google ADK	kurumsal entegrasyon	ekosistem bağı

TEYİTSİZ Bu tablo bir **okuma**, bir kıyaslama koşusu değil. Ölçtüğümüz tek çerçeve AutoGen; diğerleri kendi belgelerinden okundu. Aynı görevi beşine de koşturup token ve süre karşılaştırması yapmadık — yapılısaydı bu tablo değişebilirdi.

Resmî çok-ajan desenleri – sekizi

DESEN	05 SATIRI	NE ZAMAN
Concurrent Agents	3236	bağımsız kaynaklar paralel taranacak
Sequential Workflow	3504	adımlar birbirine bağımlı
Group Chat	3772	tartışma gerçekten gerekiyor
Handoffs	4349	devretme mantığı ajanın kendisinde
Mixture of Agents	4989	aynı soruya farklı uzmanlıklar
Multi-Agent Debate	5358	birden çok tur karşılıklı eleştiri
Reflection	5822	kalite kritik, ikinci göz lazım
Code Execution	6188	modelin yazdığı kod koşacak

KAYNAK Bu liste **core kılavuzunun kendi bölümlenmesi** (05:3206'dan itibaren), bizim tasnifimiz değil. Satır numarası verilmesinin sebebi bu: her satır kaynak metinde açılabilir.

Runtime yaşam döngüsü

```
runtime = SingleThreadedAgentRuntime()
await Agent.register(runtime, "worker", factory)
runtime.start()
await runtime.publish_message(task, topic)
await runtime.stop_when_idle()
```

`start()` mesaj işleyen döngüyü açar — ondan önce yapılan yayınlar kuyruğa girer ama işlenmez.

`stop_when_idle()` kuyruk boşalınca döner; yani bütün zincir tamamlanmış olur.

`stop()` ise **hemen** durur — işlenmemiş mesajlar kuyrukta kalır ve kaybolur.

Testlerde `stop()` kullanmak, yarısı işlenmiş bir sistemi doğru sanmanın en hızlı yoludur. Sonuç yeşil görünür çünkü assert'ler henüz gelmemiş mesajları beklemez.



AutoGen AgentChat

Ajanlar, takımlar, mesajlar, akış — kullanıma hazır katman

1. core ile ilişkisi
2. AssistantAgent ve tool döngüsü
3. Workbench ve MCP
4. Beş takım tipi
5. GraphFlow ve DAG kurma
6. On bir sonlandırma koşulu
7. Mesaj ve olay türleri
8. Bağlam, cache sınırı, durum

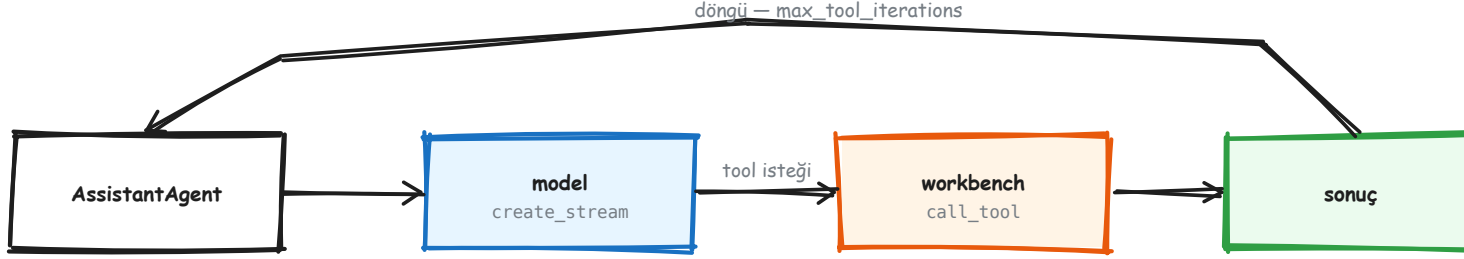
İki katman – hangisi ne zaman

	AUTOGEN_CORE	AUTOGEN_AGENTCHAT
soyutlama	aktör, mesaj, topic, abonelik	ajan, takım, görev
kim yazar	altyapı/protokol kuran	uygulama yazar
eşzamanlılık	abonelikten doğar	takım tipinden gelir
hata	yayında loglanır, kaybolur	TaskResult'ta görünür
ne zaman	kendi desenini kuruyorsan	iş yaptırıyorsan

AgentChat core'un **üstünde** duruyor, yerine değil. Bir `AssistantAgent` aslında bir `RoutedAgent` ; bir takım da arka planda bir runtime kuruyor.

Aşağı inmek her zaman mümkün ve gerektiğinde iniyoruz. Ama önce yukarıdan denemek doğru sıra: AgentChat'in çözdüğü bir problemi core'da yeniden çözmek, çoğu zaman aynı kodu daha az testle yazmak demek.

AssistantAgent ve tool döngüsü



VARSAYILAN 1: model tool sonucunu GÖRMEDEN cevap verir

ölçüldü — bizde 6'ya çekildi

ÖLÇÜLDÜ En pahalı varsayılan burada:

`max_tool_iterations=1`.

Model tool'u çağırıyor, tool koşuyor, sonuç dönüyor — ve tur **bitiyor**. Model o sonucu hiç görmüyor. Kullanıcıya giden cevap, tool'un bulduğu şeyi içermiyor.

Hiçbir uyarı çıkmıyor. Sistem çalışıyor, ajan cevap veriyor, cevap makul görünüyor — sadece tool'un getirdiği bilgi orada değil.

Bizde 6'ya çekildi. Doğru sayı göreve bağlı; doğru olmayan tek şey varsayılanı **bilmeden** bırakmak.

Bu tuzağın imzası şu: "tool çağırılıyor mu?" diye loga bakarsın, çağırılıyor. "Sonuç dönüyor mu?" diye bakarsın, dönüyor. Yine de cevap yanlış. Çünkü eksik olan çağrı değil, **ikinci model turu**.

Workbench – tool'ların toplandığı yer

`StaticWorkbench` — elindeki Python fonksiyonları. Şemaları imzadan türetiliyor.

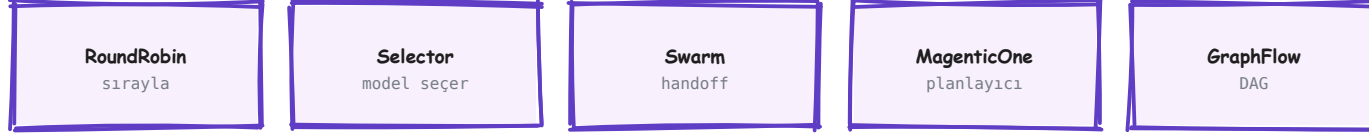
`McpWorkbench` — bir MCP sunucusunun tool'ları; stdio ile alt süreç ya da HTTP ile uzak sunucu. Ajan açısından fark yok.

KAYNAK Ajan hangi workbench'le konuştuğunu **bilmiyor**. Arayüz tek: `list_tools()` ve `call_tool()`.

Kapıyı kurmayı mümkün kılan tam olarak bu: `GatedWorkbench` de sadece bir workbench.

```
workbench = McpWorkbench(  
    StdioServerParams(  
        command="openclaw",  
        args=["mcp", "serve"],  
    )  
)  
agent = AssistantAgent(  
    "vc", model_client=client,  
    workbench=workbench,  
    max_tool_iterations=6,  
)
```

Beş takım tipi



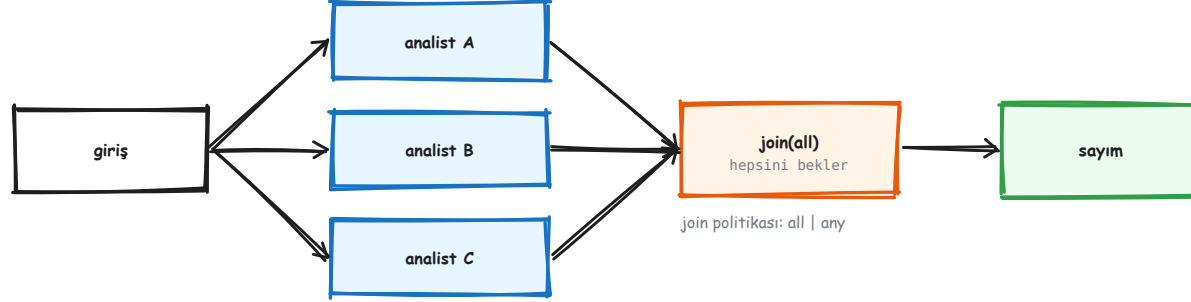
Beşi de aynı arayüz: `run()` / `run_stream()` → `TaskResult`

Taramamız *GraphFlow* kullanıyor — eşzamanlı dal + `join(all)`

TAKIM	SIRAYI KİM BELİRLER	TİPİK KULLANIM	MALİYET NOTU
RoundRobinGroupChat	sabit döngü	yazar-eleştirmen çiftleri	öngörülebilir
SelectorGroupChat	model, her turda	rolleri belirsiz tartışma	her turda ek çağrı
Swarm	ajanın kendisi (handoff)	devretme akışları	devir sayısına bağlı
MagenticOne	planlayıcı ajan	açık uçlu görevler	en pahalısı
GraphFlow	önceden çizilmiş DAG	bilinen boru hattı	ek model çağrısı yok

KAYNAK Beşi de aynı arayüzü sunuyor: `run()` / `run_stream()` → `TaskResult`. Takım değiştirmek çağıran kodu değiştirmiyor.

GraphFlow – bizim taramanın kullandığı



DiGraphBuilder ile kurulur · dallar EŞZAMANLI koşar

```

b = DiGraphBuilder()
b.add_node(intake)
for a in analysts:
    b.add_node(a)
    b.add_edge(intake, a)
    b.add_edge(a, join)
flow = GraphFlow(b.build(),
                  participants=[...])
    
```

Grafiği **önceden** çiziyorsun; sırayı model belirlemiyor. Bu, boru hattı belliyse hem daha ucuz hem daha öngörülebilir.

ÖLÇÜLDÜ Bir düzeltme: uzun süre taramanın core pub/sub kullandığını sandık ve panele o şemayı çizdik. Kod okununca çıktı — tarama `graph.py` 'deki GraphFlow'u kullanıyor, `fanin.py` 'yi **hiç çağırmıyor**.

Şimdi bunu bir test tutuyor.

On bir sonlandırma koşulu

`MaxMessageTermination` — mesaj sayısı

`TokenUsageTermination` — token bütçesi

`TimeoutTermination` — duvar saati

`TextMentionTermination` — bir kelime geçti

`TextMessageTermination` — belirli ajandan metin

`SourceMatchTermination` — belirli ajan konuştu

`HandoffTermination` — devir istendi

`StopMessageTermination` — açık dur mesajı

`FunctionCallTermination` — belirli tool çağrıldı

`FunctionalTermination` — kendi yazdığın koşul

`ExternalTermination` — dışarıdan tetiklenen

```
termination = MaxMessageTermination(20) | TokenUsageTermination(50_000)
```

KAYNAK | ile birleşenlerden **biri** yeterli, **&** ile **hepsi** gerekli.

Üretimde en az bir **sert tavan** (mesaj ya da token) olmadan takım koşturmak, faturayı modelin kararına bırakmaktır. Semantik koşullar — "TERMINATE yaz" gibi — model uymazsa çalışmaz.

Mesaj ve olay türleri

TextMessage	düz metin
StructuredMessage	pydantic şema
ToolCallRequestEvent	model tool istedi
ToolCallExecutionEvent	tool koştı
ModelClientStreamingChunkEvent	token akışı
TaskResult	bitiş + stop_reason

Ayrım: mesaj konuşmanın parçası, olay ise ne olduğunun anlatımı.

Ayrım: mesaj konuşmanın parçası, olay ise ne olduğunun anlatımı. Mesajlar bağlama girer; olaylar gözlem içindir.

KAYNAK `StructuredMessage` pydantic şeması taşıyor — serbest metin ayrıştırmak yerine tip alıyorsun. Bizim tarama sonuçları bunu kullanıyor: aday nesnesi metin değil, alanları belli bir yapı.

run ve run_stream

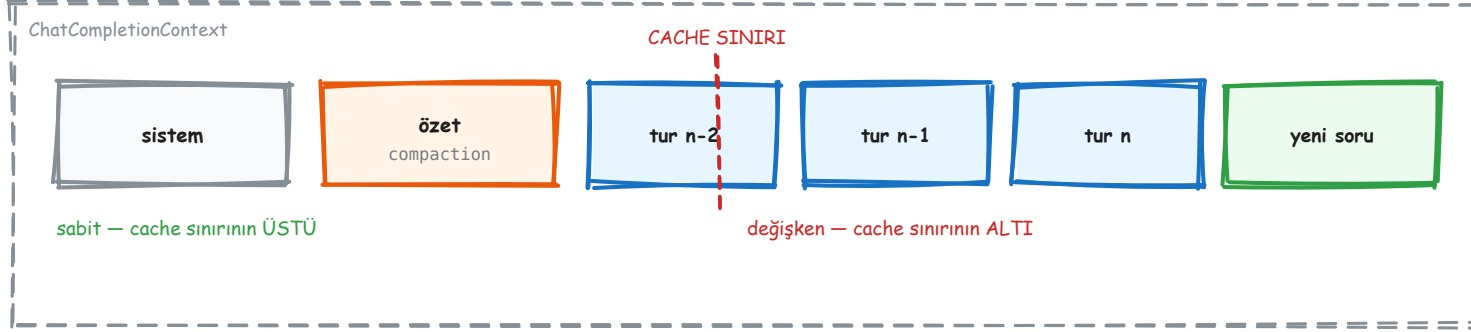
```
result = await team.run(task="...")  
print(result.stop_reason)
```

```
async for ev in team.run_stream(task="..."):  
    if isinstance(ev, TaskResult):  
        final = ev  
    else:  
        handle(ev) # ara olaylar
```

`run_stream` ara olayları verir **ve** son `TaskResult` 'ı akışın son elemanı olarak verir. Döngüde tip kontrolü yapmazsan sonucu bir olay sanıp işler, sonra da "sonuç gelmedi" diye ararsın.

ÖLÇÜLDÜ Panelimiz tam olarak bu akıştan besleniyor: her olay tipi bir mekanizma şemasına eşleniyor, ve ekrana o an koşan şey çiziliyor (Kısım IV).

Bağlam ve cache sınırı



`ChatCompletionContext` modele ne gideceğine karar veriyor. Bizimki üstüne sıkıştırma ekliyor: eşik aşılnca eski turlar bir özete iniyor, özet bağlamda kalıyor.

KAYNAK Sıralama mimari bir karar. Değişmeyen şey (sistem talimatı, tool tanımları, yetenek dizini) önde durursa prompt cache'i turlar arasında yeniden kullanılır. Değişken bir şey öne kaçarsa cache her turda bozulur ve ödeme her turda tam yapılır.

Bu yüzden "prompt'u kısalt" tavsiyesi çoğu zaman yanlış hedef. Asıl soru **neyin sabit kalabildiği**. Sabit kısım büyük olabilir; yeter ki gerçekten sabit olsun.

Durum: kaydet ve geri yükle

```
state = await team.save_state()
# ... süreç yeniden başlar ...
await team.load_state(state)
```

TEYİTSİZ Kaydedilen şey **takımın** durumu: katılımcı ajanların bağlamları, sıra bilgisi, sonlandırma sayaçları.

Workbench'in durumu **değil**. MCP oturumu geri gelmiyor; süreç yeniden başladığında yeniden kuruluyor. Tool tarafında kalıcı bir şey istiyorsan onu kendin saklaman gerekiyor.

Bunu ölçmedik — belge böyle söylüyor ve mimariyle tutarlı.

AutoGen Studio – ve neden ayrı kurduk

ÖLÇÜLDÜ `autogenstudio 0.4.2.2`, `autogen-core<0.6` pinliyor ve 0.5.7 kuruyor. Bizim proje 0.7.5 üstünde çalışıyor.

Aynı ortama kurmak projeyi **geriye** düşürürdü — ve bunu sessizce yapardı, çünkü pip uyumlu bir sürüm bulup indirir.

Bir aracı denemek için ana projenin bağımlılıklarını düşürmek, denemenin maliyetini projeye yazmaktır. Ayrı ortam beş dakika, geri alma yarım gün.

Çözüm: ayrı sanal ortam, `~/autogenstudio/.venv`. Studio 8080'de koşuyor, proje kendi 0.7.5'iyle kalıyor.

ÖLÇÜLDÜ Kurulumdan **önce** `.gitignore` 'a eklendi: `myapp/`, `autogenstudio/`, `.autogenstudio/` — appdir veritabanı API anahtarı tutuyor.

III

Ölçülen tuzaklar

Belgeden değil, çarparak öğrenilenler

1. model_info zorunluluğu
2. Sessiz varsayılanlar
3. Sessiz veri kaybı
4. Dört isim karmaşası
5. Bakım modu tezi
6. Desen seçmenin faturası

model_info – OpenAI uyumlu her endpoint'te

```
OpenAIChatCompletionClient(  
    model="deepseek-v3",  
    base_url="https://api.deepseek.com/v1",  
)  
# ValueError: model_info is required
```

ÖLÇÜLDÜ AutoGen model adını bilinen bir OpenAI modeli olarak tanıyamazsa **başlamayı reddediyor**. Sebebi makul: vision var mı, function calling destekliyor mu, JSON çıktı verebiliyor mu — bunları bilmeden ajan kurarsa yanlış varsayımla koşar.

Çözüm `model_info` 'yu elle vermek. Bizde `config.LIVE_MODEL_INFO` bunu yapıyor, bu yüzden endpoint değiştirdiğimizde bu hata artık çıkmıyor.

Bugün DeepSeek'i denerken ölçtük: tuzak **tetiklenmedi**, çünkü config zaten dolduruyordu. Hata 401'de çıktı — yani sorun anahtardı, model adı değil. Kapatılmış bir tuzağın kanıtı, başka bir hatanın önce gelmesidir.

Sessiz varsayılanlar

AYAR	VARSAYILAN	NE OLUR	NASIL FARK EDİLİR
max_tool_iterations	1	model tool sonucunu görmeden cevaplar	cevapta tool bulgusu yok
reflect_on_tool_use	False	tool çıktısı ham döner, yorumlanmaz	cevap JSON gibi görünür
sonlandırma koşulu	yok	takım tavan olmadan koşar	fatura
model_client_stream	False	akış olayı hiç yayılmaz	arayüzde token akmaz

Dördünün ortak yanı: **hiçbiri hata vermiyor**. Sistem çalışıyor, sonuç yanlış oluyor. Bir çerçevede en pahalı şey, sessizce makul görünen varsayılandır — çünkü onu aramak için önce yanlış olduğunu bilmen gerekir.

ÖLÇÜLDÜ Dördü de bizde açıkça ayarlanıyor. Varsayılanı güvenmek yerine değeri yazmak, bir sonraki sürümde varsayılan değişirse de korur.

Sessiz veri kaybı

ÖLÇÜLDÜ Yayın yolunda handler bir istisna atarsa: mesaj kaybolur, çağırın haber almaz, log'a bir satır düşer. Toplayıcı beklemeye devam eder ve sistem asılı kalır.

```
# YANLIŞ – sayaç yalnız başarıda artar
async def on_result(self, msg, ctx):
    data = parse(msg)      # patlarsa?
    self.seen += 1
    if self.seen == self.expected:
        await self.finish()
```

```
# DOĞRU – her dal sayılır
async def on_result(self, msg, ctx):
    try:
        data = parse(msg)
    except Exception as exc:
        data = Failed(str(exc))
    finally:
        self.seen += 1
        if self.seen == self.expected:
            await self.finish()
```

Bizim tarama bu yüzden **başarısız kaynakları da** sonuç nesnesinde taşıyor: `failed_sources` boş değilse rapor bunu söylüyor. "Üç kaynaktan iki sonuç" cümlesi, "iki sonuç" cümlesinden farklıdır.

Dört isim, tek karmaşa

İSİM	NE	DURUM	İMPORT SATIRI
AutoGen v0.2	eski API	terk edildi	<code>from autogen import ...</code>
AutoGen v0.4+	bugünkü mimari	bizim kullandığımız	<code>from autogen_agentchat... import ...</code>
AG2	v0.2'den çatallanan topluluk sürümü	ayrı proje	<code>from autogen import ...</code>
MAF / Agent Framework	Microsoft'un yeni birleşik çerçevesi	AutoGen'in devamı olarak konumlanıyor	ayrı paket

KAYNAK **Ayırt etme kuralı import satırında:** `from autogen import AssistantAgent` gören her örnek eski API ya da AG2'dir. Bizimki her zaman alt paket adıyla gelir: `autogen_agentchat`, `autogen_core`, `autogen_ext`.

İnternetteki örneklerin çoğu hâlâ v0.2. Bu ayrımı bilmeden Stack Overflow okumak, çalışmayan kodu kendi hatan sanmanın en hızlı yoludur.

Bakım modu – tez ve kanıtın sınırı

TEYİTSİZ Tez: AutoGen aktif özellik geliştirmeden bakım moduna geçiyor; enerji MAF'a kayıyor.

Neye dayanıyor: depo etkinliğinin şekli, Microsoft'un MAF konumlandırması, ve resmî belgelerdeki yönlendirme.

Neye dayanmıyor: bir duyuruya. Ortada "AutoGen dondu" diyen resmî bir cümle yok.

Neyi kanıtlamaz: AutoGen'in çalışmadığını. v0.7.5 üstünde kurduğumuz her şey koşuyor ve testleri geçiyor. Bakım modu *yeni özellik beklememek* demektir — *bugün kırık* demek değil.

Karar açısından anlamı: yeni bir proje bugün başlıyorsa bu bilgi seçime girer. Çalışan bir proje için taşınma gerekçesi değildir; taşınmanın maliyeti, beklenmeyen özelliğin değerinden büyüktür.

Desen seçmenin faturası

%63.7

aynı görev, aynı ajanlar — yalnız orkestrasyon değişince ortaya çıkan token farkı

DESEN	MESAJ	LLM	TOOL	TOKEN
SelectorGroupChat	8	5	2	204
GraphFlow	11	7	3	270
RoundRobinGroupChat	9	6	2	274
Swarm (handoff)	14	7	4	334

ÖLÇÜLDÜ poc/kiyas.py — aynı görevi beş desenle koşturuyor, yalnız orkestrasyon değişiyor. Anahtar yoksa replay modunda çalışıyor, yani sayılar **tekrarlanabilir**.

Ödenen şey zekâ değil **yönlendirme özerkliği**. Swarm her devirde bağlamı yeniden kuruyor; Selector tek bir seçim çağrısıyla idare ediyor.

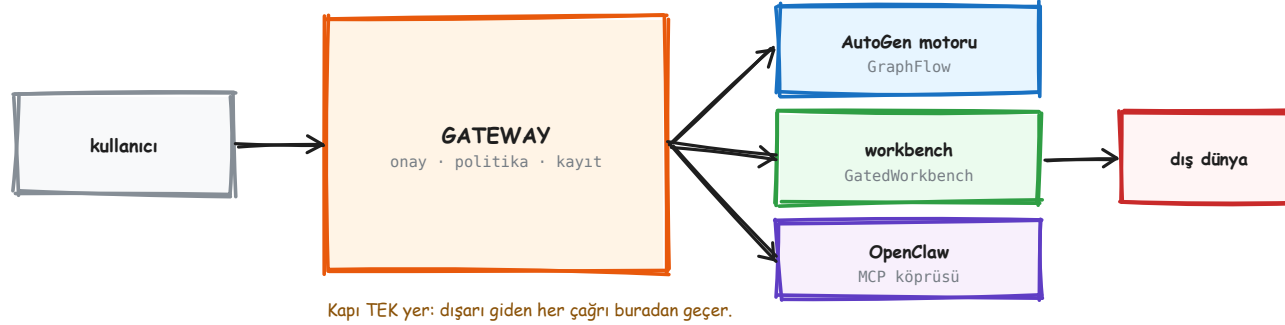
IV

VC Gateway – bizim kurduğumuz

OpenClaw'ın biçimi, AutoGen'in motoru

1. Neden iki sistemi birleştirdik
2. Maliyet gradyanlı huni
3. Dört ilke
4. Onay kapısı ve bulduğumuz hata
5. /openclaw kaçış kapağı
6. İki yönlü MCP köprüsü
7. Canlı mekanizma paneli
8. Belge araması, testler, hukuki sınır

Ne kurduk ve neden



AutoGen bir kapı kavramı sunmuyor: bir ajanın dış dünyaya yaptığı çağrıyı durdurup insana sormak istiyorsan, o mekanizmayı kendin kuruyorsun.

OpenClaw o mekanizmayı sunuyor ama ajan döngüsü sıradan — takım, graf, sonlandırma gibi soyutlamaları yok.

Birleştirdik: **biçim** OpenClaw'dan (kapı, politika, kayıt), **motor** AutoGen'den (GraphFlow, akış, sonlandırma).

ÖLÇÜLDÜ 214 testle başladı, bugün daha fazla. Testlerin bir kısmı çerçeveyi değil **bizim varsayımlarımızı** tutuyor — taramanın GraphFlow olduğu, onay metninin id taşıdığı, engelli alan adlarının koşulsuz olduğu gibi.

Bir üçüncü taraf kütüphaneyi test etmek genelde israftır. Kendi varsayımını test etmek değildir — çünkü kayan şey kütüphane değil, senin ona dair inancındır.

Huni – maliyet gradyanlı

KADEME	NE YAPAR	ADAY SAYISINA ETKİSİ	MALİYET
1 · toplama	anahtarsız kaynaklar, hız sınırlı	yüzlerce girer	≈ 0
2 · kural filtresi	tarih, dil, tekilleştirme	kabaca yarıya iner	≈ 0
3 · ucuz model	sinyal mi, değil mi	onlara iner	düşük
4 · orta model	zenginleştirme, yapılandırma	aynı kalır	orta
5 · güçlü model	yalnız finalistler	birkaç tane	yüksek

İlke: **pahalı olan en sona**. Bir aday beşinci kademeye geldiğinde onu oraya taşıyan dört ucuz karar zaten verilmiştir. Ters sıralama aynı sonucu 10-20 kat pahalıya üretir.

ÖLÇÜLDÜ Sayılar **funnel** nesnesinde taşınıyor ve rapor kaç adayın hangi kademede elendiğini söylüyor. Bu, huninin çalışıp çalışmadığını **görünür** kılıyor: ikinci kademe hiçbir şey elemiyorsa filtre yanlış yazılmıştır.

Dört ilke

1 · Her cevap kaynağını söyler. Söyleyemiyorsa cevap değildir. Belge aramasında her isabet dosya adı ve satır numarasıyla geliyor.

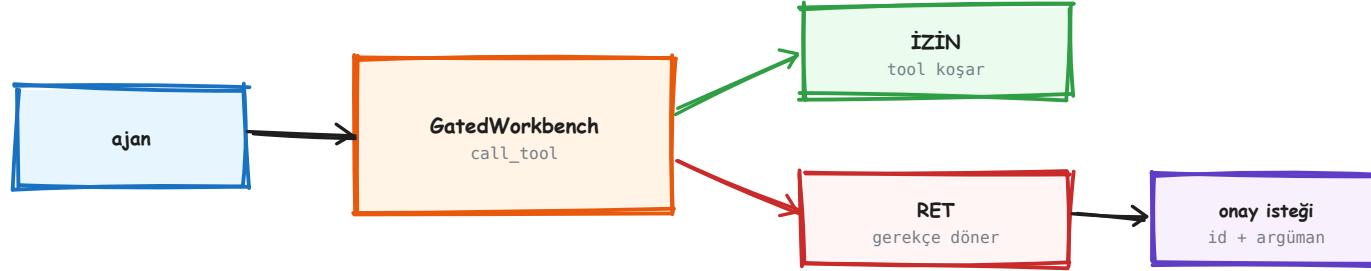
2 · Sıfır sonuç bir açıklama borcudur. Boş liste "bulunamadı" demek değil; "şu kaynaklar şu yüzden düştü, şu filtre şu kadarını elledi" demektir.

Dördü de koda gömülü, slogan değil: her birinin bir testi var. Bir ilke test edilemiyorsa o bir niyet beyanıdır, bir kural değil.

3 · Pahalı olan en sona. Huni bunun uygulaması.

4 · Dışarı giden her çağrı kapıdan geçer. Tek istisnası, yazarın kendi elleriyle yazdığı `/openclaw` — ve o istisna bilinçli, belgeli ve sınırlı.

Onay kapısı



Onay metni id'yi TAŞIMALI — düşerse arayüz düğmeyi çizemez.

Bu hatayı ölçtük: testi önce eski koda karşı düşürdük.

Kapı workbench katmanında: ajan `call_tool` çağırıyor, kapı araya giriyor. İzin verirse tool koşuyor; vermezse ajana bir **gerekçe** dönüyor — sessiz bir düşürme değil.

Gerekçenin ajana dönmesi önemli: model neden reddedildiğini görüp başka bir yol deneyebiliyor.

ÖLÇÜLDÜ **Bulduğumuz hata:** çağırın kendi gerekçesini verince, varsayılan metnin **yerine** geçiyordu — ve varsayılan metin onay id'sini taşıyordu.

Sonuç: arayüz, onaylanabilir ve bekleyen bir istek için "onay yolu yok" çiziyordu. İstek duruyordu, düğme yoktu.

Düzeltilme: id her zaman **eklenir**, asla değiştirilmez.

/openclaw – kaçış kapağı

```
/openclaw adın ne  
→ OpenClaw'ın kendi ajanına gider
```

```
/openclaw skills.status  
→ RPC metodu olarak gider
```

Ayrımı **nokta** yapıyor: token'da nokta varsa metot, yoksa cümle. Basit bir kural, ama tahmin etmiyor — belirsizse metot saymıyor.

Cümleler `chat.send` ile gidiyor (`idempotencyKey` zorunlu — ilk denemede öğrendik), cevap ise `chat.history` yoklanarak alınıyor.

ÖLÇÜLDÜ Çünkü `chat.send` cevabı değil `{runId, status:"started"}` döndürüyor. Koşu sürerken geçmişte boş asistan satırları görünebiliyor; onları boş sayıp beklemek gerekiyor.

Bilerek kapıyı atlıyor: bunu yazan zaten yazarın kendisi. Ama YASAK metotlar yine reddediliyor. Uygulama çok kullanıcı olursa bu `peer=="local"` varsayımı kırılır — ve o gün bu slayt bir hata raporudur.

İki yönlü MCP köprüsü

Onlardan bize: `McpWorkbench`, OpenClaw'ın tool'larını AutoGen ajanına veriyor. Ajan bunları kendi tool'larından ayırt etmiyor.

Bizden onlara: kendi tool'larımızı bir MCP sunucusu olarak sunuyoruz; OpenClaw ajanı onları çağırabiliyor.

Aynı protokol iki yöne de çalışıyor — MCP'nin asıl kazandırdığı bu.

Aynı ilke OpenClaw'ın kendi SecretRef kararında da var (Kısım VI): sırrı okuyabilen her bileşen, sırrın saldırı yüzeyinin parçasıdır.

ÖLÇÜLDÜ **Çizdiğimiz sınır:** `~/.openclaw/openclaw.json` sır tutuyor ve kodumuz onu **okumuyor**. Yalnız `openclaw` ikilisiyle konuşuyoruz.

Bu bilinçli: yapılandırma dosyasını okumak, o sırları bizim süreç sınırimıza taşımak olurdu. Okumadığın şey, sızdıramadığın şeydir.

Canlı mekanizma paneli

ÖLÇÜLDÜ Sorulan her soruda ekranda **o an hangi AutoGen mekanizmasının koştugu** çiziliyor: bağlam kurulumu → model çağırısı → token akışı → tool isteği → kapı → tool koşumu → döngü → bitiş.

Her mekanizma gerçek sınıf adını ve modül dosyasını söylüyor. Bir test o dosyaların diskte gerçekten var olduğunu doğruluyor — yani panel kayarsa test kırılıyor.

Sunucu tarafı `stages.py`: mekanizma kataloğu tek kaynakta, ve alt süreçten gelen aşamalar `##STAGE` satırlarıyla taşınıyor.

Panelin dürüstlük kuralı: core şeridi `routed: 0` yazıyor — çünkü sohbet turu core pub/sub'ı gerçekten kullanmıyor.

Uydurulmuş bir kutuyu yakmak, panelin öğrettiği her şeyi değersizleştirirdi. Bir öğretme aracının ilk görevi doğru olmaktır; ikinci görevi etkileyici olmak.

Grafiği kaldırıp yerine **yalnız o an koşan mekanizmanın kendi şemasını** çizmeye geçtik — akış çubuğu her şeyi aynı anda gösterdiği için hiçbir şeyi göstermiyordu.

Belge araması – ve bir kirlenme

`docs/` 1.18 MB. Prompt'a sığmaz, aranması gerekir. TF-IDF seçildi, embedding değil — üç gerekçeyle:

- indeks ikinci bir bayatlayabilir şeydir, ve bayat indeks eski belgeden kendinden emin cevap verir
- 675 bölümü gömmek endpoint'in ayakta olmasını gerektirir — *belge* aramasının model sağlayıcısıyla düşmesi saçmadır
- bu belgeler tam terimlerle dolu (`model_info` , `ClosureAgent`), ki bu leksik skorlamanın en güçlü olduğu yer

ÖLÇÜLDÜ Ölçülen kirlenme: 63 KB'lık kaydedilmiş bir blog sayfası `docs/` 'a düşünce idf ağırlığı kaydı ve kendi tuzaklar sayfamız, sahibi olduğu bir sorguda ilk dörtten çıktı.

Düzeltilme: yalnız numaralı seri indeksleniyor. `docs/` 'a okumak için bir şey koymak artık aramayı bozmuyor.

Arama kalitesinin, birisi okumak için bir şey kaydettiği için bozulması, sahip olmaya değer bir hata modu değildir.

Anahtar olayı ve play.py

ÖLÇÜLDÜ Bugün ölçüldü: `.env`'deki anahtar DeepSeek'e karşı **401** aldı. Endpoint'e ulaşıldı (1.52 sn), istek biçimi doğruydı, `model_info` tuzağı tetiklenmedi — reddedilen şey yalnızca anahtardı.

Ek gözlem: 78 karakterlik bir anahtar, DeepSeek biçimine (`sk-` + ~32) uymuyor.

`play.py` bunun için yazıldı: boru hattının **kendi** config'ini kullanan tek atışlık bir erişim testi. Yeşil çıkarsa `/api/chat` de çalışır.

İki kuralı var:

- anahtarı asla basmaz — uzunluk + kısa önek maskesi
- eksik yapılandırmayla **ateş etmez**

Yarım yapılandırmayla — URL tahmin ederek, ya da bir anahtarı ait olmadığı endpoint'e göndererek — koşan bir test, hiç testten kötüdür. Sonucu yorumlanamaz, ve yan etkisi gerçektir.

Test disiplini

Testlerin bir kısmı çerçeveyi değil **bizim varsayımlarımızı** tutuyor:

- taramanın GraphFlow olduğu
- onay metninin id taşıdığı
- docs/ 'a düşen yabancı dosyanın indekslenmediği
- engelli alan adlarının koşulsuz olduğu
- panelin adını verdiği modüllerin diskte var olduğu

ÖLÇÜLDÜ **Kural:** bir hatanın testi, düzeltmeden **önce** eski koda karşı düşürülür. Kırmızı olduğunu görmeden düzeltme yapılmıyor.

Onay id'si hatasında bunu uyguladık: test önce kırmızıydı, sonra düzeltme geldi. Ters sırada yazılan bir test, düzeltmeyi değil **kendini** doğrular — çünkü zaten geçen bir dünyada yazılmıştır.

Hukuki sınır – koşulsuz

Engelli alan adları: LinkedIn · Facebook · Instagram · X/Twitter · Crunchbase · PitchBook

Bu liste **koşulsuz**. Yapılandırmayla açılmıyor, bir bayrakla atlanamıyor, "sadece test için" istisnası yok.

KAYNAK Gerekçe hukuki: bu sitelerin kullanım şartları otomatik erişimi yasaklıyor. Bir VC aracı için toplanan veri, **toplanma biçimi yüzünden** kullanılamaz hale gelirse hiç toplanmamış sayılır.

`test_policy.py` bu sınırı tutuyor — yani birisi ileride gevşetmeye çalışırsa test kırılıyor.

Bu yüzden agent-reach'in Twitter/Reddit kazıma yüzeyi bizde doğrudan kullanılamıyor. Ekleme kararı verilmeden önce bu çatışmanın çözülmesi gerekiyor — "sonra bakarız" bir çözüm değil.

Şekiller neden elle çizilmiş görünüyor

ÖLÇÜLDÜ Ölçülen tuzak: `feTurbulence` + `feDisplacementMap` tarayıcıda titrek görünüyor, ama Chrome PDF'e basarken filtreyi **sessizce düşürüyor**. Filtreli ve filtresiz çıktı piksel piksel aynıydı — ve hiçbir uyarı yoktu.

Çözüm: titreşimi yol verisine **pişirmek**. Buradaki her dikdörtgen, köşeleri kaydırılmış gerçek bir `path`; render anında hiçbir şeyin işbirliği yapması gerekmiyor.

Üç kural var, üçüncüsü en çok atlanan:

1. Köşeler ıskalar — her tepe bir iki piksel kayar
2. Kenarlar yay çizer — her kenar hafif şişkin bir eğri
3. **Her kontur iki kez çizilir**, farklı sapmayla — kalem üstünden geçmiş gibi. Excalidraw görünümünün asıl kaynağı bu.

KAYNAK Sapma tohumlu bir PRNG'den geliyor; aynı belge her derlemede bayt-eş çıkıyor.

Kendini her derlemede yeniden karan bir diyagram, her derlemeyi gözden geçirmede bir değişiklik gibi gösterir.

V

OpenClaw'ın içi

Canlı RPC ile ölçüldü — iddia değil, sayı

1. Yüzeyin büyüklüğü
2. Skill kademeli açığa çıkarma
3. Sürüm önbelleği
4. Beş bellek katmanı
5. İki şeritli geri çağırma
6. Bağlam motoru ve sıkıştırma
7. Zamanlama
8. Protokol, eklenti, node

Yüzey – ölçülen sayılar

NE	SAYI	NASIL ÖLÇÜLDÜ
Gateway RPC metodu	351	canlı RPC
Tool	44	<code>tools.catalog</code>
Tool grubu	14	aynı çağrı
Tool profili	4	aynı çağrı
Kurulu skill	74	<code>skills.status</code>
Model-görünür skill	40	aynı çağrı

ÖLÇÜLDÜ Hepsi **çalışan bir gateway'e sorularak** alındı, belgeden okunarak değil. Bu ayrım önemli: belge sürümler arasında kayabilir, çalışan sistem kayamaz.

74 kurulu skill'in yalnız 40'ının model-görünür olması bir yapılandırma kararı — geri kalanı yüklü ama kapalı. Yani "kurulu" ile "etkin" farklı iki sayı, ve ikisini karıştırmak yüzeyi iki kat büyük göstermek olurdu.

Kademeli açığa çıkarma

PROMPT'TA NE VAR



gövdeler diskte bekler:



...74 tane

Gövde yalnız model `read` ile isteyince yüklenir — kademeli açığa çıkarma.



ölçülen fark:

%93 token tasarrufu

Mekanizma basit: prompt'a yalnız skill'lerin **indeksi** giriyor — ad ve tek satır açıklama. Gövdeler diskte duruyor.

Model bir skill'e ihtiyaç duyduğunda `read` ile gövdesini çekiyor. İhtiyaç duymadıklarının gövdesi hiç yüklenmiyor.

ÖLÇÜLDÜ %93 tasarruf: diskteki gövde boyutlarının toplamıyla prompt'a giren indeks boyutu karşılaştırılarak hesaplandı.

Bunun ikinci bir faydası var: model 74 talimatı aynı anda görmediği için dikkati dağılmıyor. Yani kazanç yalnız token değil, **isabet**.

Aynı fikir tool katmanında da var (Tool Search — Kısım VI). İki yerde bağımsız olarak ortaya çıkması, bunun bir numara değil bir **desen** olduğunu gösteriyor.

Sürüm önbelleği – sha256 sinyali

ÖLÇÜLDÜ Skill dosyalarının sha256'sı tutuluyor. Hash değişmediyse dosya yeniden okunmuyor.

Bu, önbelleğin **geçerlilik sinyali**: zaman aşımı değil, içerik hash'i. Bir dosya bir yıl değişmezse bir yıl yeniden okunmuyor; bir saniye önce değiştiyse hemen yeniden okunuyor.

Zaman tabanlı önbellek "ne zaman bayatlar" sorusunu **tahmin etmeye** çalışır ve her zaman ya çok erken ya çok geç tahmin eder. İçerik tabanlı önbellek o soruyu ortadan kaldırır.

Bizde karşılığı yok ama olması gereken bir yer var: `docs_index` her içe aktarımda bütün korpusu yeniden işliyor. Dosya hash'i tutulsa aynı kazanç orada da olurdu.

Sistem prompt'u koddan değil, dosyalardan derleniyor

Ajanın davranışı kaynak kodda değil, çalışma alanındaki markdown dosyalarında tanımlı. Runtime her oturumda bunları derleyip prompt'a koyuyor:

`AGENTS.md` — işletim talimatı, pazarlıksız kurallar

`SOUL.md` — ses tonu ve üslup

`IDENTITY.md` — ad, kimlik

`USER.md` — kullanıcı modeli

`MEMORY.md` — küratörlü uzun dönem bellek

`BOOTSTRAP.md` — yalnız yepyeni çalışma alanında

ÖLÇÜLDÜ Sınırlar ölçülü: dosya başına **20.000** karakter, toplam **60.000**. Aşan dosya kesiliyor ve kesildiği **uyarılıyor** (`bootstrapPromptTruncationWarning` , varsayılan `always`).

`memory/GG-AA-YY.md` günlükleri bu derlemeye **girmiyor** — yalnız `memory_search` ile isteğe bağlı geliyor.

Enterprise açısından değerli ayrım: **davranış dosyada, zorlama runtime'da**. Bir ekip asistanın üslubunu ve kurallarını kod değiştirmeden düzenleyebiliyor; ama izin, sandbox ve onay hâlâ runtime'ın elinde — ve markdown'la gevşetilemiyor.

Beş bellek katmanı

KATMAN	YÜZEY	KİM YAZAR	NE ZAMAN ENJEKTE
Instructions	AGENTS.md	yalnız insan yazar	her oturum
Curated core	MEMORY.md	kapılı konsolidasyon	her oturum
Episodic	memory/GG.md	ajan çalışırken	hiç — aranır
Prospective	SQLite	intent tool	tetiklenince
Review	DREAMS.md	dreaming	hiç — insan okur

Sınır ikinci ile üçüncü arasında: episodic'ten curated'a hiçbir şey kapıdan geçmeden çıkmaz.

KAYNAK Sınır ikinci ile üçüncü arasında. **Curated** küçük, her oturumda bağlamda, ve yalnız kapılı konsolidasyonla yazılıyor. **Episodic** büyük, ekleme dostu, ve yalnız arama yoluyla erişilebiliyor. Episodic'ten curated'a hiçbir şey kapıdan geçmeden çıkmıyor.

Geri çağırma – iki şerit

Şerit 1 — her zaman açık, sıfır model çağırısı.

Deterministik skor: $\text{önem} \times \text{tazelik} \times \text{ilgi}$. Eşiği (≥ 0.72) geçen tetikleyici bağlama enjekte ediliyor.

Şerit 2 — tırmanma.

Birinci şerit yetmezse model çağırısı içeren arama devreye giriyor.

"Deterministik kapılar, içlerinde model yargısı." Bu cümle OpenClaw'ın bellek belgesinin tasarım ilkelerinden biri. Skarlama, eşikler, uygunluk ve yaşam döngüsü **kod**; model yalnız kodun izin verdiği yerde konuşuyor.

ÖLÇÜLDÜ Ayrımın maliyeti şu: normal turların çoğu bellek için **hiç model çağırıyor**. Model yalnız gerçekten dil yargısı gerektiğinde, ve deterministik kodun çizdiği sınırların içinde devreye giriyor.

Bağlam motoru ve sıkıştırma

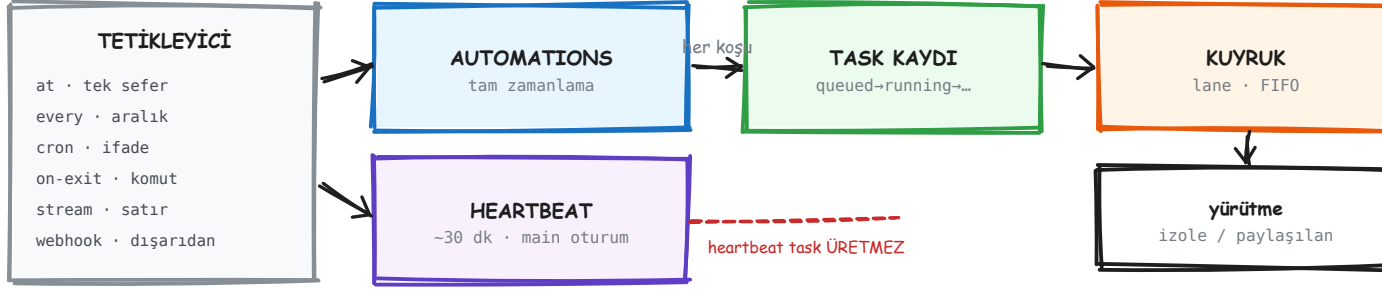
Bağlam motorunun **dört yaşam döngüsü noktası** var ve eklenti olarak değiştirilebiliyor — yani bağlam kurma stratejisi çekirdeğe gömülü değil.

KAYNAK Sıkıştırma bir kuralı asla bozmuyor: **tool çağrısını sonucundan ayırmıyor**. Ayırsaydı model, cevabını hiç görmediği bir çağrı yapmış gibi görünürdü — ve çoğu sağlayıcı bu şekli geçersiz sayar.

KAYNAK Hatalar cevabı bloklamıyor: cevap yolundaki her bellek/bağlam adımının bir timeout'u, bir geri düşüşü veya ikisi var.

Çöken bir bellek altsistemi geri çağırma kalitesini düşürür; bir turu asla yemez. Bu, üretimde koşan bir asistan için pazarlık konusu olmayan bir özellik.

Zamanlama – önce haritanın kendisi



Sıra soldan sağa: ne tetikler → kim karar verir → ne kaydedilir → nasıl serileşir → nerede koşar.

Soldan sağa: ne tetikler → kim karar verir → ne kaydedilir → nasıl serileşir → nerede koşar.

"Zamanlanmış iş" tek bir mekanizma değil. OpenClaw bunu **altı ayrı parçaya** bölmüş ve her biri farklı bir soruyu cevaplıyor:

Automations — tam zamanlama

Heartbeat — yaklaşık, bağlamli yoklama

Tasks — ne olduğunun kaydı

Task Flow — çok adımlı dayanıklı akış

Hooks — yaşam döngüsü olaylarına tepki

Standing orders — kalıcı talimat

KAYNAK Belgenin kendi karar rehberi tek soruyla ayırıyor: *tam zamanlama mı gerekiyor, esnek mi?* Tam ise Automations, esnek ise Heartbeat.

Bu ayrım önemsiz görünüyor ama arkasında bambaşka iki yürütme modeli var — sonraki slayt.

Beş zamanlama türü

TÜR	BAYRAK	NE YAPAR
<code>at</code>	<code>--at</code>	tek seferlik zaman damgası (ISO 8601 ya da <code>20m</code> gibi görelî)
<code>every</code>	<code>--every</code>	sabit aralık — <code>10m</code> , <code>1h</code> , <code>1d</code>
<code>cron</code>	<code>--cron</code>	5 ya da 6 alanlı cron ifadesi, isteğe bağlı <code>--tz</code>
<code>on-exit</code>	<code>--on-exit</code>	izlenen bir komut çıkınca bir kez — tur yıkımından sağ çıkar
<code>stream</code>	<code>--stream-command</code>	uzun ömürlü bir komutun stdout/stderr satırlarından

ÖLÇÜLDÜ Son ikisi zamana **hiç** bakmıyor. `on-exit` ve `stream` olay güdümlü — "zamanlayıcı" kelimesi ikisini de yanlış anlatıyor.

Zaman dilimi kuralı: dilimsiz zaman damgaları **UTC** sayılıyor. `--tz` yalnız `at` ve `cron` ile geçerli.

KAYNAK **Yük dağıtma:** saat başına denk gelen tekrarlı işler (dakika `0`, saat joker) kendiliğinden **5 dakikaya kadar** kaydırılıyor — aynı anda uyanan yüz işin yük tepesi oluşturmaması için. `--exact` ile kapatılıyor.

Bu, ölçekte fark eden ama kimsenin akla getirmediği türden bir varsayılan.

Cron'un OR tuzağı

KAYNAK Cron ifadeleri **croner** ile ayrıştırılıyor. Ayın-günü ve haftanın-günü alanlarının **ikisi de** joker değilse, **croner ya biri ya öteki** eşleştğinde tetikliyor — ikisi birden değil.

Bu standart Vixie cron davranışı, yani hata değil. Ama neredeyse herkesin yanlış okuduğu bir davranış.

```
# Niyet: "ayın 15'i, ama yalnız Pazartesiye"  
0 9 15 * 1
```

```
# Gerçek: her ayın 15'inde 9'da  
#           VE her Pazartesi 9'da
```

ÖLÇÜLDÜ Ayda 0–1 kez yerine **ayda 5–6 kez** çalışıyor.

Çözüm: **croner**'in **+** değiştiricisi (`0 9 15 * +1`), ya da bir alanı zamanlamaya bırakıp diğerini işin kendi içinde kontrol etmek. Bir kurumsal asistanda "ayda beş kez rapor gönderdi" bir arıza kaydıdır.

Koşul gözcüsü – zamana değil duruma bağlanmak

Bir **event trigger**, `every` / `cron` / `stream` zamanlamasının üstüne başsız bir **koşul betiği** ekliyor. Zamanlama geldiğinde önce betik koşuyor; yük ancak `fire: true` dönerse çalışıyor.

Betik kalıcı bir `trigger.state` taşıyor — yani bir önceki değerlendirmeyi hatırlıyor. Böylece **değişim** tespit edilebiliyor, yalnız durum değil.

KAYNAK Bu, "kurumsal karşılık"ın tam olarak oturduğu yer: *limit aşıldığında, itiraz üç günü geçtiğinde, skor eşiği düştüğünde* — takvim değil, **iş kuralı**.

```
// her 30 sn'de bak, ama yalnız DEĞİŞİNCE tetikle
const s = await tools.call('exec', {...});
json({
  fire: s !== trigger.state?.status,
  message: `CI: ${trigger.state?.status} -> ${s}`,
  state: { status: s },
});
```

KAYNAK Ve bir güvenlik sınırı: `koşul betikleri` `cron.triggers.enabled: true` istiyor, çünkü gözetimsiz koşan bir betik ayrı bir güven sınıfı. Varsayılan kapalı.

İki zamanlayıcı – ve neden ikisi birden var

BOYUT	AUTOMATIONS	HEARTBEAT
zamanlama	tam (cron, tek seferlik)	yaklaşık — varsayılan 30 dk
oturum bağlamı	taze (izole) veya paylaşılan	tam main-session bağlamı
task kaydı	her zaman yaratılır	asla yaratılmaz
teslimat	kanal, webhook, ya da sessiz	main session içinde satır içi
en uygun	raporlar, hatırlatmalar, arka plan işleri	gelen kutusu, takvim, bildirim

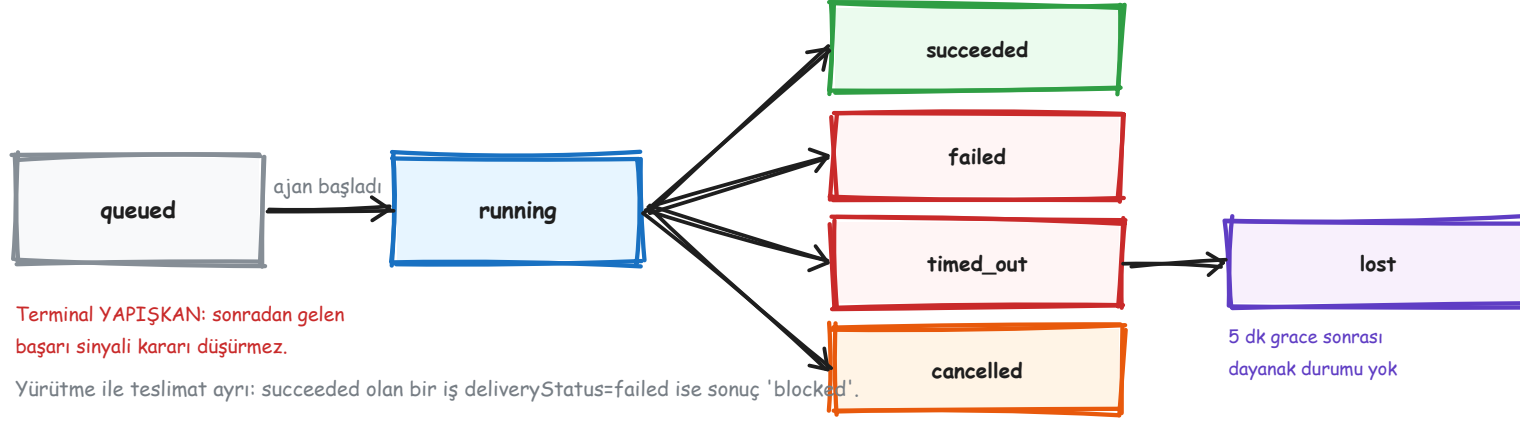
Neden ikisi? Çünkü "her 30 dakikada gelen kutusuna bak" ile "her sabah 9'da raporu gönder" farklı şeyler istiyor.

Birincisi **bağlam** istiyor — asistanın konuştuklarını bilmesi lazım. İkincisi **izolasyon** istiyor — sohbetin gürültüsü rapora karışmasın.

KAYNAK Heartbeat meşgulken **kendiliğinden erteleniyor**: ana kuyruk ya da automation işi doluyorsa, aynı ajan için başka bir cevap koşuyorsa, ya da hedef oturumda aktif/kuyrukta iş varsa.

Yani periyodik yoklama, gerçek işin önüne geçmiyor.

Task ledger – kayıt, zamanlayıcı değil



Beş terminal durum, biri ölçüm.

Tasks are records, not schedulers — automations ve heartbeat için *ne zaman* koşacağına karar verir; task'lar *ne olduğunu* izler.

KAYNAK Task yaratanlar: ACP koşuları, subagent spawn'ları, **her** automation koşusu, CLI işlemleri, medya üretimi.

KAYNAK **Yaratmayanlar:** heartbeat turları, normal sohbet turları, doğrudan `/komut` cevapları.

Saklama: terminal kayıtlar **7 gün**, `lost` olanlar **24 saat**, sonra otomatik budanıyor.

Üç kural ki tasarımı bu yapıyor

① Yürütme ile teslimat ayrı.

KAYNAK Bir subagent task'ı `succeeded` kalabilir ama `deliveryStatus` `failed` olabilir. O zaman sonuç `blocked` oluyor — **failed değil**.

Gerekçesi belgede: *"Bu, tamamlanmış sonucu korur; çocuk yürütmesini yanlışlıkla başarısız diye raporlamak yerine."*

② Terminal yapışkan.

KAYNAK Bir task terminal olduktan sonra gelen yaşam döngüsü sinyalleri onu **düşüremiyor**. Operatör iptal ettiyse, sonradan gelen bir başarı sinyali kararı değiştirmiyor.

③ `lost` çalışma-zamanı farkında.

KAYNAK Her kaynak için kanıt standardı farklı: ACP için *yalnız* canlı bir in-process tur kanıt sayılıyor — kalıcı oturum metadata'sı yetmiyor. Automation için önce runtime, sonra dayanıklı koşu geçmiş kontrol ediliyor.

Ve en güzel cümlesi: *"Çevrimdışı CLI denetimi, kendi boş in-process durumunu otorite saymaz."* Yani **kanıtın yokluğu, yokluğun kanıtı değil** — bu tek cümle bir mühendislik olgunluğu göstergesi.

Yoklama yanlış şekil

Task belgesinin en yüksek getirili cümlesi:

Tamamlanma itmeli (push-driven): ayrılmış iş bittiğinde doğrudan bildirebilir ya da isteyen oturumu / heartbeat'i uyandırabilir — **bu yüzden durum yoklama döngüleri genelde yanlış şekildir.**

Bir ajana "iş başlat, sonra bitti mi diye sor" dedirtmek doğal geliyor. Ama her yoklama bir model turu, ve iş beş dakika sürüyorsa on tur boşa gidiyor.

Doğru şekil: iş bitince **o** seni uyandırıyor.

ÖLÇÜLDÜ Bizim tarafta karşılığı yok — henüz.

pipeline/ 'da ayrılmış iş kavramı yok, her şey istek içinde bitiyor.

Atlas'ta olacak: uzun süren bir sorgu ya da rapor, kullanıcıyı bekletmeden koşup bitince haber vermeli.

Kuyruk – koşular nasıl çakışmıyor

KAYNAK Gelen bütün otomatik cevap koşuları küçük bir **lane-aware FIFO** kuyruktan geçiyor. Amaç çakışmayı önlemek: oturum durumu, loglar, CLI stdin paylaşılan kaynaklar.

İki katmanlı:

- `session:<key>` lane → **oturum başına tek** aktif koşu
- global `main` lane → toplam paralellik `maxConcurrent` ile sınırlı

Bu, Atlas'a doğrudan taşınacak: aynı kullanıcının iki isteği birbirinin bağlamını bozmamalı, ama farklı kullanıcılar birbirini beklememeli. Ayrımı **lane** yapıyor.

ÖLÇÜLDÜ Ölçülen varsayılanlar: `main` lane `min(16, max(8, CPU çekirdeği))`, `subagent` lane **8**, yapılandırılmamış lane'ler **1**.

Ve bir kullanıcı deneyimi ayrıntısı: yazıyor göstergesi kuyruğa girer girmez tetikleniyor — koşu sırasını beklerken bile kullanıcı sistemin uyandığını görüyor.

Task Flow – çok adımlı ve dayanıklı

Tek bir arka plan işi **task**. Çok adımlı bir boru hattı **flow**. Flow'un kendi durumu, JSON durumu, **revizyon sayacı** ve bağlı task kayıtları var.

İki kip:

managed — plugin kodu sürücü; adımları açıkça ilerletiyor

mirrored — ayrılmış ACP/subagent spawn'ları için otomatik

KAYNAK **Revizyon sayacı neden var:** her değişiklik flow'un beklenen revizyonunu taşıyor. Bayat bir yazma, daha yeni durumu ezmek yerine **revizyon çakışması** olarak reddediliyor.

Ve iptal: iptal istendikten sonra yeni çocuk task kabul edilmiyor; flow, aktif çocuk kalmayınca **cancelled** olarak sonlanıyor.

Flow'lar gateway restart'ını **sağ atlatıyor**; task'lar ayrılmış işin birimi olarak kalıyor. Kurumsal karşılığı: "üç günlük bir itiraz sürecini takip et" işinin sunucu yeniden başlayınca kaybolmaması.

Zamanlamadan Atlas'a ne taşınır

MEKANİZMA	NEDEN DEĞERLİ	ATLAS'TA KARŞILIĞI
Kayıt ile zamanlayıcının ayrılması	"ne zaman" ile "ne oldu" farklı sorular	denetime giden şey kayıt; zamanlama operasyon
Yürütme $\neq$ teslimat	biten iş, teslim edilemedi diye başarısız sayılmıyor	"rapor üretildi ama e-posta gitmedi" ayırt edilebilir
Terminal yapışkanlığı	geç gelen sinyal kararı bozmuyor	iptal edilen bir sorgu geri dirilmiyor
lost 'un kanıt standardı	kanıtın yokluğu, yokluğun kanıtı değil	denetimde "bilmiyoruz" diyebilmek
İtmeli tamamlanma	yoklama döngüsü token yakıyor	uzun sorgu kullanıcıyı bekletmiyor
Lane'li kuyruk	oturum izolasyonu + kontrollü paralellik	departmanlar birbirini beklemiyor
Koşul gözcüsü	takvim değil iş kuralı	"limit aşıldığında" tetikleme

Gateway protokolü

WebSocket üstünde JSON-RPC benzeri çerçeveleme. Bağlantı bir **el sıkışma** ile başlıyor: rol (`operator / node`), istenen kapsamlar, istemci kimliği.

ÖLÇÜLDÜ 351 metot ölçüldü. Aileler: `chat.*`, `session.*`, `tools.*`, `skills.*`, `audit.*`, `device.*`, `node.*`, `config.*`.

ÖLÇÜLDÜ Ölçerken çarptığımız: `workspace.read_file` diye bir metot **yok**. Uydurmuştuk; canlı gateway “unknown method” dedi.

Bir RPC yüzeyini belgeden değil **kendisinden** sormanın değeri bu: var sandığın metot yoksa ilk çağrıda öğrenirsin, üç saat sonra değil.

Eklenti ve yetenek sistemi

ÖLÇÜLDÜ 161 extension, 22 paket. Yetenek türleri: tool sağlayıcı, kanal, model sağlayıcı, bağlam motoru, bellek sağlayıcı, sıkıştırma sağlayıcı.

Yani **çekirdeğin her ilginç parçası değiştirilebilir** — sıkıştırma bile. Bu, mimari bir duruş: çekirdek bir çerçeve, bir ürün değil.

Anahtar ayırım: *skill* ile *plugin* aynı şey değil.

Skill = markdown talimat + isteğe bağlı betik. Modelin *okuduğu* şey.

Plugin = kod. Runtime'a *yetenek ekleyen* şey.

KAYNAK Bir plugin tool ekler; bir skill o tool'un **ne zaman** kullanılacağını anlatır.

İkisini karıştırmak, yetenek eklemekle talimat eklemeyi karıştırmaktır. Yeni bir tool yazman gereken yere skill yazarsan model yapamayacağı bir şeyi anlatan bir metin okur.

Node'lar – cihaz yetenekleri

Gateway merkezde; **node**'lar cihaz yeteneklerini sunuyor — macOS, iOS, Android, headless.

`node.invoke` ile çağrılıyorlar.

Güven **pairing** (eşleşme) ile kuruluyor: node bağlanır, bekleyen bir talep doğar, bir operatör onaylar.

Yetki, bağlanan şeyin **ne yapabildiğine** göre türetiliyor — sabit bir role göre değil. Bu, Kısım VI'daki "kapsam parametreden türetilir" fikrinin donanım tarafındaki karşılığı.

KAYNAK **İlginç kısım:** eşleşme onayı, düğümün **bildirdiği komut listesinden** ek kapsam türetiyor. `system.run`, `fs.listdir`, `browser.proxy` gibi komutlar `operator.admin` istiyor; sıradan komutlar istemiyor.

VI

Enterprise: Atlas

Resmî repodan ne alınır, ne alınmaz

1. Resmî repo ölçüleri
2. Tez: mekanizma taşınır, güven modeli taşınmaz
3. Üç kontrol eksenî
4. Onay = donmuş plan
5. Denetim ve dürüst sınır beyanı
6. Yetki ve dış içerik sınırı
7. Tool Search, bellek, sırlar, telemetri
8. ALINMAYACAKLAR
9. Atlas'ın şekli ve ilk üç iş

Kaynak – resmî repo

NE	DEĞER
depo	github.com/openclaw/openclaw
commit	01cc7106
boyut	558 MB
extension	161
paket	22
belge dosyası	764

ÖLÇÜLDÜ Diskte: ~/Desktop/adapted/harnesses/openclaw .

Bu kısımdaki her iddianın altında bir dosya adı var; 764 belgenin enterprise açısından anlamlı ~20'si okundu.

Hoş tesadüf: OpenClaw'ın kendi tehdit modeli dosyasının adı THREAT-MODEL-ATLAS.md . Oradaki ATLAS, MITRE'nin yapay zekâ tehdit çerçevesi — bizim Atlas'la ilgisi yok. Ama **biçimi** doğrudan kopyalanacak cinsten.

Tez – ne taşınır, ne taşınmaz

MEKANİZMA
kopyalanabilir

OpenClaw TEK bir güvencilen operatör etrafında tasarlanmıştır.
Atlas çok kullanıcı ve karşılıklı güvenmeyen departmanlar içerecektir.

Mekanizmalar taşınır. Güven modeli yeniden kurulur.

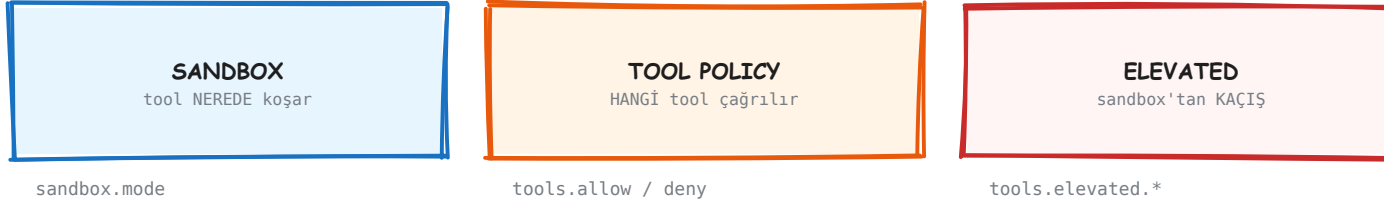
GÜVEN MODELİ
kopyalanamaz

OpenClaw'dan alınacak şey ajan döngüsü değil, **ajanı kuşatan kontrol düzlemi** — ve daha da değerlisi, o düzlemin kendi sınırlarını dürüstçe beyan etme alışkanlığı.

İkinci yarı birincisinden kıymetli. OpenClaw'ın belgeleri sürekli şunu yapıyor: bir mekanizmayı anlatıyor, sonra "bu şunu **kanıtlamaz**" diye kendi iddiasını daraltıyor.

Bir kredi bürosunda asistanı öldüren şey mekanizmanın olmaması değil — **olduğu sanılan bir mekanizmanın denetim toplantısında çökmesi.**

Üç kontrol eksenı



deny HER ZAMAN kazanır · allow doluysa gerisi kapalı

Ama: tool policy ADA göre filtreler — exec'in İÇİNİ görmez.

"write'ı kapattık, artık read-only" YANLIŞTIR.

KAYNAK docs/gateway/sandbox-vs-tool-policy-vs-elevated.md . Üç ayrı soru, üç ayrı mekanizma — ve karıştırılmaları en yaygın yapılandırma hatası.

Son satır özellikle önemli: **"write tool'unu kapattık, artık read-only"** cümlesi yanlıştır. `exec` serbestse shell üstünden yazma zaten mümkündür. Salt-okunur bir rol istiyorsan `group:runtime` 'ı da kapatman gerekir.

Roller tool listesi değil, grup adı

OpenClaw'da **13 grup** **ÖLÇÜLDÜ**: `group:runtime`,
`group:fs`, `group:web`, `group:memory`, `group:sessions`,
`group:agents`, `group:media`, `group:nodes` ...

Politika bunlarla yazılıyor: `allow: ["group:fs",
"group:memory"]`.

KKB karşılığı (öneri):

`group:musteri-verisi`
`group:kredi-sorgu`
`group:rapor`
`group:dis-erisim`

Rol tanımı birkaç grup adı olur, kırk satırlık bir tool listesi değil.

Kazanç bakımında: yeni bir tool eklendiğinde kırk rol dosyası güncellenmez. Tool doğru gruba girer, roller kendiliğinden doğru kalır.

Onay = dondurulmuş plan



Dosya onaydan sonra değişirse de reddedilir — kaymış içerik koşmaz.

Bizim `approval.py` bugün argüman hash'i TUTMUYOR. Açık iş.

KAYNAK `docs/tools/exec-approvals.md`. Naif bir onay akışında onay ile çalıştırma arasında bir TOCTOU boşluğu vardır: kullanıcı gördüğü şeyi onaylar, çalışan başka bir şey olabilir.

KKB'de somut anlamı: "şu TCKN için kredi notu sorgula" onayı *o TCKN'ye* bağlanır. Onay alındıktan sonra parametre değiştirilemez; değiştiyse istek düşer, sessizce yeni parametreyle koşmaz.

ÖLÇÜLDÜ Bizim `approval.py` bugün argüman hash'i **tutmuyor**. Açık iş — Kısım VI'nın sonundaki listede ikinci sırada.

Denetim: iki hat gerekiyor

OPERASYONEL

best-effort · yalnız metadata
30 gün · 100.000 satır
kuyruk dolarsa DÜŞER, koşu sürer

UYUM ARŞİVİ

kayıpsız · senkron
denetçiye gösterilir
yazılamazsa KOŞU DÜŞER

OpenClaw yalnız solu yapıyor ve bunu açıkça söylüyor:

"Bir satırın yokluğu hiçbir şey kanıtlamaz."

KKB'nin ihtiyacı sağ taraf. Ayrımı baştan yapmak ucuz, sonradan şema göçü.

KAYNAK OpenClaw'ın denetim kaydı içerik tutmuyor: prompt, mesaj gövdesi, tool argümanı, URL, komut çıktısı — hiçbirisi. Kimlikler `hmac-sha256:v1:<keyId>:<digest>` olarak çıkıyor.

Ve kendi sınırını söylüyor: *"Bu korelasyondur, anonimleştirme değildir — veritabanını okuyabilen anahtarı da okur ve aday ham kimlikleri pseudonym'lere karşı test edebilir."*

Yetki: metot kapsamı yalnızca ilk kapı

8 kapsam var **ÖLÇÜLDÜ**: `read`, `write`, `admin`, `pairing`, `approvals`, `questions`, `talk`, `talk.secrets`. Ama asıl fikir tabloda değil, başlıkta.

Kapsam parametreden türetiliyor. `agent` metodu normal turlar için `write`, `/reset` için `admin` istiyor. `chat.send` write-scoped, ama içindeki `/config set` komutu — çağırının chat kapsamı ne olursa olsun — `admin` istiyor.

KAYNAK Yetki yükseltme yasak. Bir cihazı onaylamak yalnız çağırının *zaten sahip olduğu* kapsamaları basabiliyor.

Ve sessiz genişleme yok: daha geniş rol isteyen bir yeniden bağlanma, yeni bir **bekleyen yükseltme talebi** doğuruyor.

Atlas'a taşınacak: "bu kullanıcı bu metodu çağırabilir mi" yetmez — "**bu parametrelerle** çağırabilir mi" gerekir.

Dış içerik = veri, talimat değil



<<<EXTERNAL_UNTRUSTED_CONTENT id="a3f9...">>>

id rastgele ÇÜNKÜ sabit olsaydı içerik kendi kapanışını yazıp sarmalayıcıdan çıkardı.

ÖLÇÜLDÜ src/security/external-content.ts , 468 satır.
Sayımlar kaynaktan: **14** şüpheli desen, **22** özel token literalı, **28** homoglif eşlemesi.

Desen eşleştirme injection'ı **engellemiyor** — içerik yine işleniyor, desenler yalnız loglanıyor. Doğru karar: tespit bir sinyal, bir savunma değil. Savunma sarmalayıcının kendisi.

Tool Search – büyük katalog, küçük prompt

Model bütün tool şemalarını görmüyor. Sınırlı bir **yetenek dizini** görüyor, sonra `search → describe → call` yapıyor.

KAYNAK Dizin 18.000 karakterle sınırlı, ada göre sıralı, ve **cache sınırının üstünde**. Kullanıcı mesajı, turbaşı tahminler ve güvenilmeyen MCP metadata'sı dizine **girmiyor** — girse cache her turda bozulurdu.

Ve fail-closed: politika dışı bir tool **aramada çıkmıyor**. Gizlemek yetmez; bulunamaz olmalı.

KAYNAK Kod köprüsü izole bir Node alt sürecinde: **boş environment, dosya sistemi yok, ağ yok, alt süreç yok**. Alt süreç plugin implementasyonlarını, MCP istemci nesnelerini veya **sırları** tutmuyor.

Her gerçek çağrı köprüden Gateway'e geri dönüyor ve normal politika / onay / hook / log akışından geçiyor.

Bellek: güvenlik sınırı yazma yolunda

Belleğin içerik düzeyinde taranması zehirlenmiş olguları güvenilir biçimde yakalayamaz — bu yüzden **yazma anında köken** zorunlu kılınır ve terfi yapısal olarak kapıya bağlanır.

Köken sınıfı **kapalı bir küme** ve SQLite sütununda — modelin düzyazıyla yazamayacağı bir yerde:

owner · agent · untrusted · system

KKB'de karşılığı doğrudan: asistanın bir müşteri kaydından okuduğu şeyi kalıcı olgu sanıp başka bir bağlamda kullanması. Buna karşı savunma içerik taraması değil, **şemada zorunlu köken sınıfı**.

Sınıflandırma muhafazakâr: belirlenemeyen köken dışsalsa **untrusted** sayılıyor, **asla owner varsayılmıyor**.

KAYNAK Ve döngü önleme: bellekten bağlama enjekte edilen içerik yapısal olarak işaretleniyor ve yeni bellek olarak **yeniden çıkarılmıyor**.

"Yüz kez hatırlanan bir olgu tek bir olgu olarak kalır."

Sırlar ve telemetri

SecretRef + egress sentinel. Loglar, SDK yapılandırması ve hata nesneleri gerçek anahtarı değil `oc-sent-v1-...` görüyor; gerçek değer istek süreçten çıkmadan hemen önce yerine konuyor.

KAYNAK Bilinmeyen sentinel-şekilli bir değer **ağ etkinliğinden önce fail-closed**: çözülmemiş sentinel sağlayıcıya ileilmektense istek hiç gönderilmiyor.

KAYNAK Ve göç bir **kapı**: `secrets audit --check` temiz değilse göç bitmemiştir.

Telemetri: içerik yok, boyut var. Prompt metni dışa aktarılmıyor. Ama aktarılanlar zengin:

```
system_prompt_chars, tool_definitions_count,  
tool_definitions_chars, request_bytes,  
time_to_first_byte_ms
```

KAYNAK Yani "prompt'ta ne kadar tool tanımı vardı" sorusu **prompt saklanmadan** cevaplanıyor.

Bir bankada telemetri sistemine prompt metni akıtmak, veri sınıflandırma politikasını sessizce delen en yaygın yoldur.

Dayanıklılık: compaction sonrası nöbetçi

KAYNAK Rolling döngü tespiti varsayılan **kapalı** — flagship modellerde nadiren gerekiyor. Ama compaction sonrası nöbetçi **açık**.

Kırdığı zincir şu: bağlam taşması → sıkıştırma → aynı döngü → taşma. Sıkıştırma-yeniden denemesinden sonra kısa bir pencerede aynı `(tool, args, result)` üçlüsü tekrarlanırsa koşu iptal ediliyor.

KAYNAK İnce ayar iki yönlü: `exec` için hash **oynak** metadata'yı (süre, PID, cwd) yok sayıyor — yoksa aynı komut hep farklı görünürdü. Giden mesajlarda ise tersi: oynak id'ler **çıkartılıyor** ki iki farklı "gönderildi" aynı görünmesin.

İki varsayılanın **farklı** olması bilinçli: agresif olan kapalı, ucuz ve yüksek getirili olan açık.

Tehdit modeli yaşayan bir belge

THREAT-MODEL-ATLAS.md , 561 satır, MITRE ATLAS taktiklerine göre düzenli. Her tehdit için **sabit bir tablo şeması**:

ATLAS ID · açıklama · saldırı vektörü · etkilenen bileşenler · **mevcut azaltmalar** · **artık risk** · öneriler

Beş güven sınırı ve altı veri akışı diyagramda tanımlı. Katkı için ayrı bir belge var — yani canlı tutuluyor.

KAYNAK Dikkat çeken dürüstlük: bazı tehditlerde "**Mevcut azaltmalar: Yok**" yazıyor. Bilinen ama kapatılmamış riskler gizlenmiyor; "artık risk: düşük" gerekçesiyle yazılıyor.

Atlas'a taşınacak: bu tablo şeması, ilk gün. KKB'de zaten bir güvenlik komitesi vardır ve o komiteye gidilecek belge budur. "Artık risk" sütununu boş bırakmamak, denetimde en çok işe yarayan alışkanlık.

Yumuşak yönlendirme, sert kontrol değildir

Destenin en kısa ve en çok işe yarayan cümlesi, OpenClaw'ın kendi güvenlik belgesinden:

Sistem prompt'undaki güvenlik kuralları yumuşak yönlendirmedir. Zorlama; kanal erişim denetimi, tool politikası, sandbox kapsamı ve — geçerliyse — açık çalıştırma onaylarından gelir.

Yani `AGENTS.md`'ye “müşteri verisini dışarı gönderme” yazmak bir **kontrol değil**. Model çoğu zaman uyar; uymadığı gün hiçbir şey onu durdurmaz.

Tersi de doğru ve daha az söyleniyor: tool politikası doğru kurulmuşsa prompt'taki kural zaten gereksizdir.

KAYNAK Ve OpenClaw bunu ekliyor: sandbox **opt-in**. Varsayılan olarak kapalı olan bir kontrolün var olması, uygulandığı anlamına gelmiyor.

Atlas için pratik ayrım:

- prompt'a yazılan → *niyet*
- tool politikası, sandbox, onay, kapı → *kontrol*

Denetim toplantısında gösterilecek olan ikinci liste. Birincisi bir belge, bir güvence değil.

ALINMAYACAKLAR

NE	OPENCLAW NE DİYOR	ATLAS'TA NEDEN YETMEZ
<code>operator.read</code> ile veri ayrımı	"düşmanca çok-kiracılı izolasyon sınırı değildir"	departmanlar birbirinin sorgusunu görmemeli — gerçek per-user authz gerekir
multi-user sahiplik / presence	"kullanılabilirlik özelliğidir, güvenlik sınırı değil"	kimlik tool politikasına girmeli, yalnız arayüze değil
denetim kaydı	"kayıpsız bir uyum arşivi değildir"	denetçi "kayıp olabilir" cevabını kabul etmez
exec approvals	"per-user auth sınırı değildir"	kazayı azaltır; kötü niyetli operatörü durdurmaz
SecretRef sentinel	"süreç izolasyonu değildir"	gerçek değer aynı süreçte — HSM/vault + süreç ayrımı ayrıca gerekir
sandbox <code>docker.binds</code>	"docker.sock bağlamak host kontrolünü verir"	bind'ler sandbox'ı deler; gözden geçirme konusu olmalı
161 extension / sohbet kanalları	—	WhatsApp/Telegram bir kurumsal asistanın yüzeyi değil

Tek cümlelik ayrım

Mekanizma taşınır.

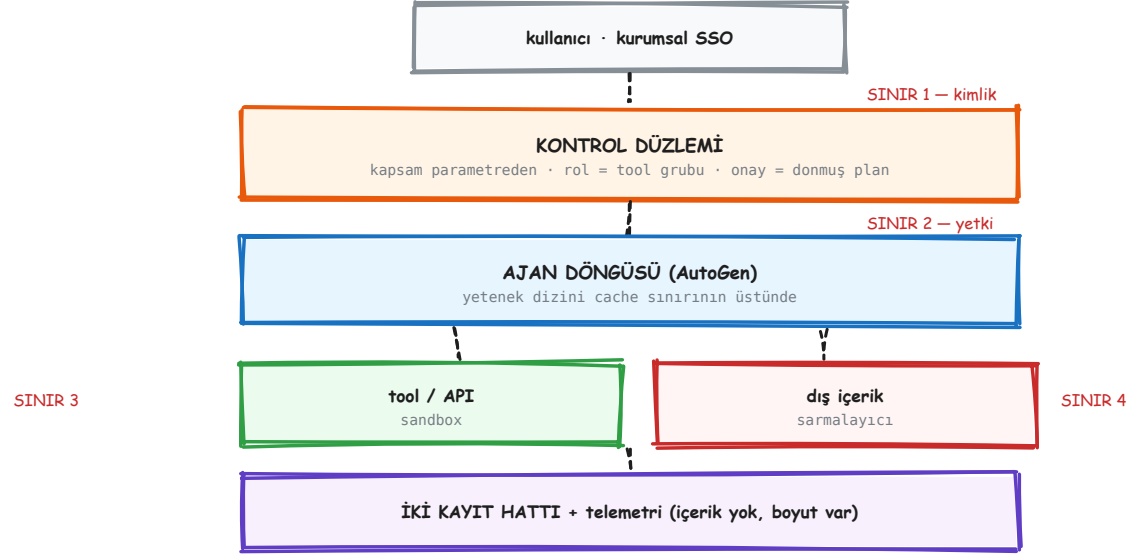
Güven modeli taşınmaz — yeniden kurulur.

OpenClaw tek bir güvenilen operatörün etrafında tasarlanmıştır. Bütün "bu bir güvenlik sınırı değildir" cümleleri buradan geliyor: o modelde zaten herkes güvenilir, dolayısıyla ayrım bir kolaylık.

Atlas çok kullanıcı ve karşılıklı güvenmeyen departmanlar içerecek. Aynı cümleler orada birer açık olur.

OpenClaw'ın kendi cevabı da bu: gerçek ayrım gerekiyorsa **ayrı gateway'ler** çalıştırın. Atlas'ta bu, gateway çoğaltmak değil, kimliği kontrol düzlemine gerçekten sokmak demek.

Atlas'ın şekli



Alınan mekanizmalar, KKB'ye göre yeniden kurulmuş güven modeliyle.

Bellek bu diyagramda ayrı bir kutu değil, **her sınırdan geçen bir sütun**: her yazılan olgu köken sınıfını taşır, ve **untrusted** olan hiçbir şey terfi edemez.

İlk üç iş

#	İŞ	SÜRE	NEDEN BU SIRADA
1	Dış içerik sarmalayıcı	~1 gün	en küçük iş, en büyük tekil koruma — ve bizde hiç yok : docs_index sonuçları ve dış metinler bağlama düz giriyor
2	Onayı plana bağlama	~1-2 gün	<code>approval.py</code> argüman hash'i tutmuyor; onay sonrası değişiklik fark edilmiyor. Testi kolay: onayla, argümanı değiştir, reddedilmeli
3	İki hatlı kayıt ayrımı	~2-3 gün	şimdi ucuz, sonra şema göçü. Uyum hattının tek sert kuralı: yazılamazsa koşu düşer

ÖLÇÜLDÜ Üçü de bugünkü `pipeline/` 'a prototiplenebilir. Ardından sırada: cache sınırı disiplini (maliyet kazancı) ve bellek köken sınıfı (şema kararı — geç kalınırsa pahalı).

Geriye kalan üç cümle

Seksen slayttan sonra taşınmaya değer olanlar:

1. AutoGen'de örtük hiçbir şey yok. "Kim tetikledi" sorusunun cevabı her zaman senin yazdığın satırdır — bu hem gücü hem yükü.
2. Bir çerçevede en pahalı şey, sessizce makul görünen varsayılandır. `max_tool_iterations=1` hata vermez, sadece yanlış cevap verir.
3. Bir mekanizmanın değeri, ne yaptığı kadar **neyi yapmadığını söylemesindedir**. OpenClaw'ın belgeleri bunu yapıyor; Atlas'ın alması gereken ilk alışkanlık bu.

Kaynak künyesi

Belgeler

- `docs/05` AutoGen core kılavuzu (resmî, birebir)
- `docs/08` AgentChat kılavuzu (resmî, birebir)
- `docs/06` ölçülen tuzaklar
- `docs/09` çerçeve karşılaştırması
- `docs/13` OpenClaw teknik analizi
- `docs/14` protokoller ve farklar
- `docs/15` gateway mimarisi
- `docs/16` enterprise ilham — bu destenin Kısım VI'sı

PDF'ler

- `agentchat-kilavuzu.pdf` — 34 sayfa, bütün yüzey
- `openclaw-ici.pdf` — 12 sayfa, ölçülmüş içi
- `autogen-openclaw-kilavuzu.pdf`

Kod

- `pipeline/` — gateway, graph, stages, kapı
- `docs/diagrams/rough.py` — bu şekilleri çizen kalem
- `docs/diagrams/make_slides.py` — bu desteyi üreten script